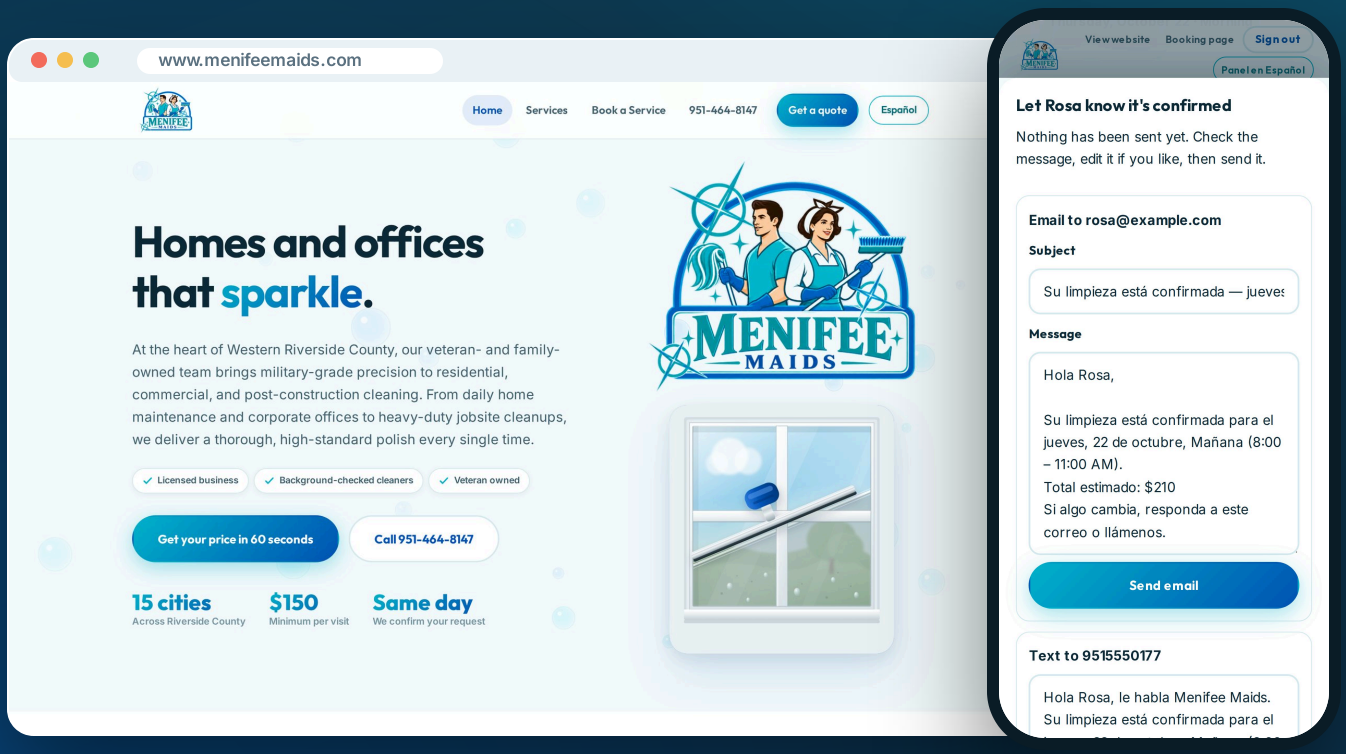
TECHNICAL PROJECT REPORT · CLIENT ENGAGEMENT

Menifee Maids: Cloud Booking & Operations Platform

A bilingual, mobile-first web platform that takes cleaning requests online, runs the business from a phone, collects card payments and keeps every staff device in sync. Designed, built and deployed end to end on Microsoft Azure.

Azure Static Web Apps | Azure Functions | Cosmos DB | Blob & Queue Storage | Communication Services | Stripe | GitHub Actions | JavaScript · Node.js



Esmeralda Cabrera Ventura

Cloud & Full-Stack Developer

Live at www.menifeemaids.com

AT A GLANCE

A real business, live in production

I designed, built and deployed the complete digital platform for **Menifee Maids**, a third-party client: a family-owned cleaning company serving fifteen cities in Western Riverside County, California: online booking, a shared staff dashboard, customer messaging and card payments, on Azure's free and pay-per-use tiers.

7
Azure services integrated, plus Stripe, GitHub Actions and custom-domain DNS

13 + 5
HTTP API endpoints, plus scheduled and queue-driven background functions

~13.3k
Lines of application code across front end, API and background jobs

2
Fully supported languages (English & Spanish), ~360 translated interface strings

130+
Automated end-to-end checks run against the real API code before each release

$0 fixed
Infrastructure designed around free tiers; usage-based costs measured in cents

What I delivered

A customer website with instant pricing and online booking · an owner and helper dashboard that works on any phone and syncs across devices · Stripe card payments with automatic reconciliation · customer emails and texts in the customer's language · job photos delivered by email and private link · background reminders, archiving and customer history · a custom domain with verified email sending.

My role

Sole developer on the engagement: requirements, solution architecture, cloud provisioning, front-end and API development, payments integration, CI/CD, DNS and email deliverability, security, test design, production troubleshooting and vendor support escalation. I used AI pair-programming to accelerate coding and review, while I made the decisions, managed the cloud environment and shipped every release.

Contents

1 The business problem & goals
2 Product tour
3 Value delivered: the operations hub
4 Architecture & key decisions
5 Delivery approach
6 Building the cloud foundation
7 Engineering deep dives
8 Troubleshooting case studies
9 Testing & quality
10 Deployment & DevOps
11 Security & compliance
12 Operations & costs
13 Skills demonstrated
14 What's next
15 Build this for your business

From text messages and a paper calendar to a real system

Most small cleaning businesses in the area run on phone calls, text threads and memory: double bookings, missed reminders, no revenue visibility. And nearly every competitor is English-only, even though nearby cities have large Spanish-speaking populations.

Four requirements that shaped every design decision

1

Book without a phone call

Customers request a cleaning in English or Spanish, see an accurate price before committing, and pick from the owner's real open days.

2

Never lose a request

Every booking is stored durably the moment it's accepted, even if a downstream service is having a bad day.

3

Run the business from a phone

The client's owner is usually on a job site. Every workflow (confirm, reschedule, complete with photos, get paid) works one-handed on a phone.

4

Keep running costs near zero

A two-person business can't absorb a large cloud bill. The architecture leans on free tiers and pay-per-use services, guarded by a $5 budget alert.

Stakeholders & constraints

WHO	WHAT THEY NEED	CONSTRAINT I DESIGNED FOR
Customers	Fast quote, easy booking, clear confirmation, simple way to pay, photos of the finished work	Mixed English/Spanish audience; mostly on phones
Client's owner	One place for requests, schedule, payments, revenue and customer messages	Works from a phone, often with weak signal
Client's helper	Today's jobs, addresses, notes and a way to close out work with photos	Must not see owner-only settings or payment configuration
Business	Professional brand, trustworthy payments, records for taxes	Tight budget; compliance (privacy, SMS consent, review rules)

2 · PRODUCT TOUR

The customer experience

A fast, bilingual, mobile-first site that turns a visitor into a booked request in about a minute, with no account, no phone call and no surprises on price.

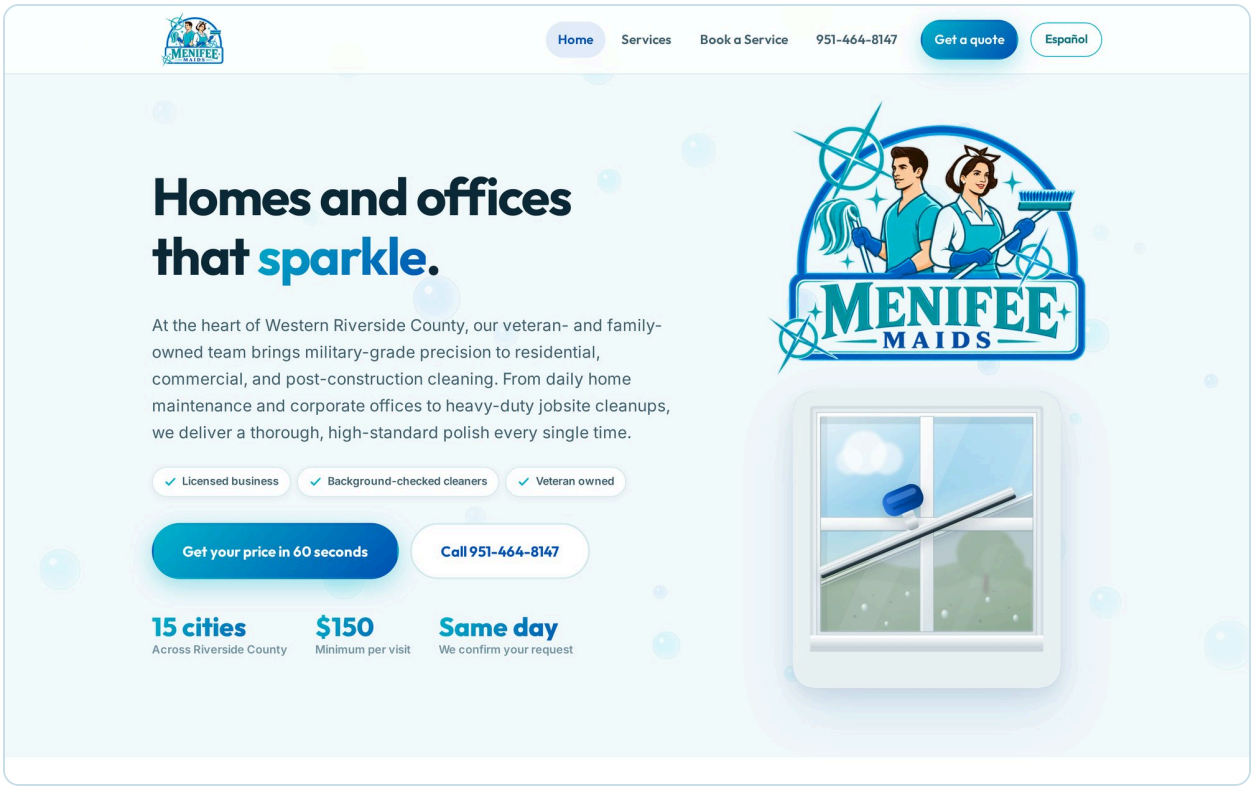


Figure 2.1 · The live home page. Brand colors come straight from the logo; the full menu stays visible on every screen size, and every page has a Spanish counterpart.

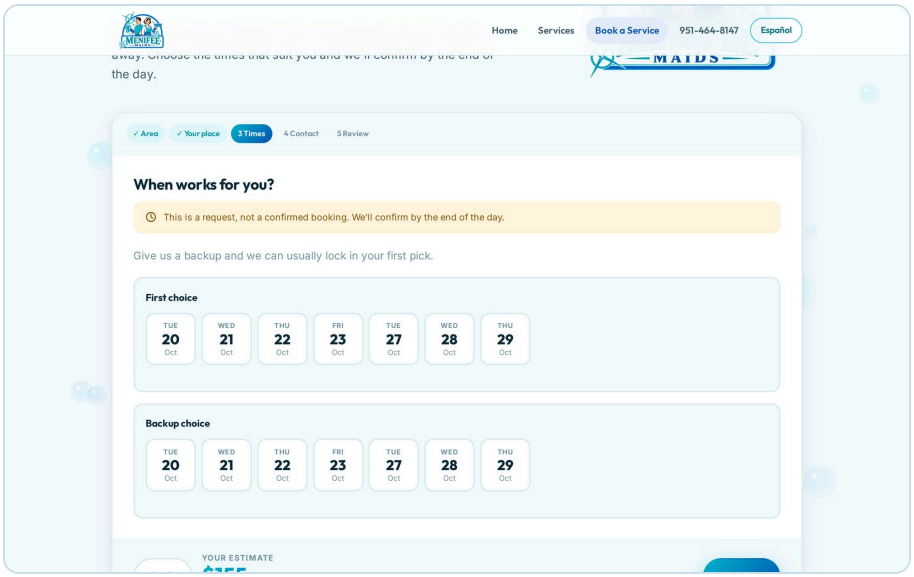


Figure 2.2 · Booking: customers pick a first choice and a backup from the owner's **real** open days, loaded live from the cloud calendar.

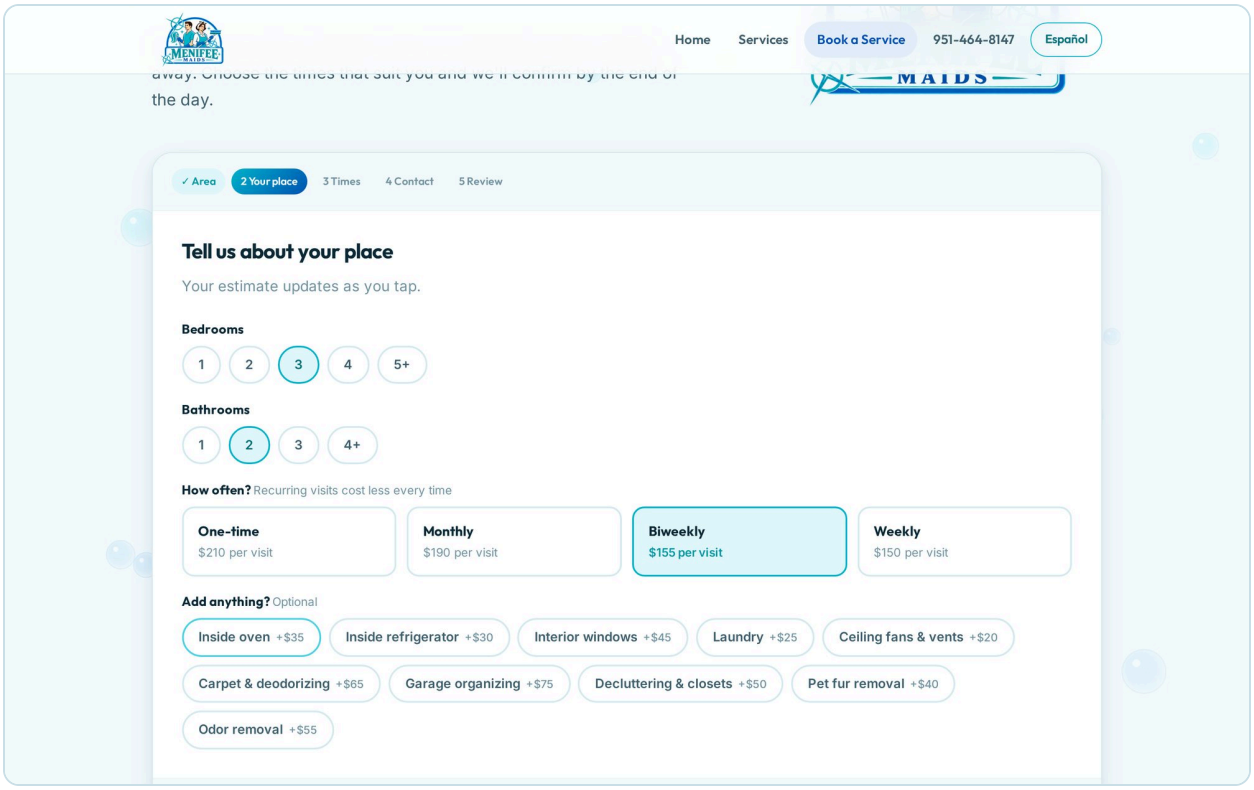
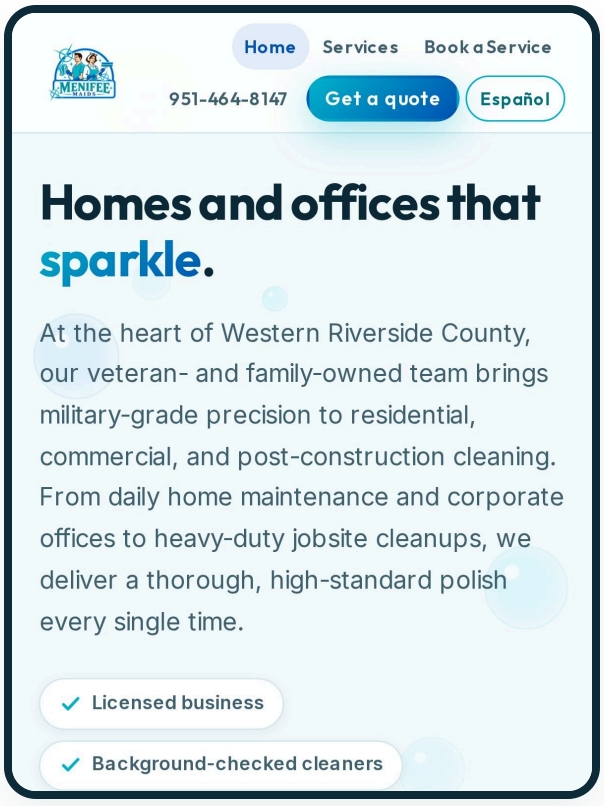
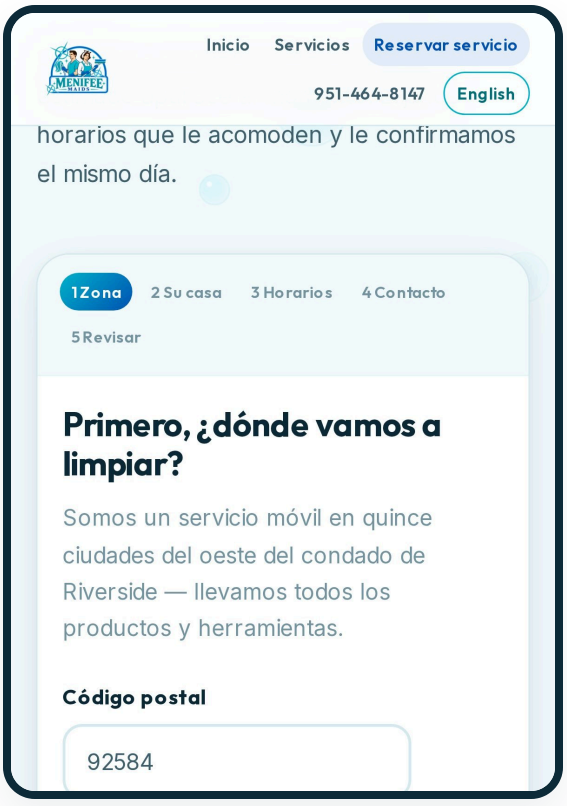


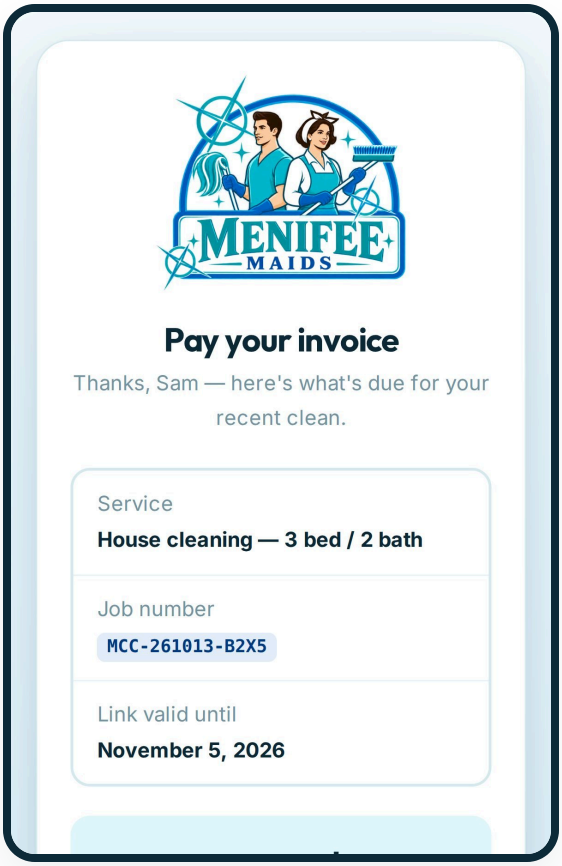
Figure 2.3 · Instant pricing updates as rooms, frequency and add-ons change. Savings are shown in dollars, not percentages.



2.4 · Phone layout, full menu visible.



2.5 · Booking in Spanish, using regional vocabulary.

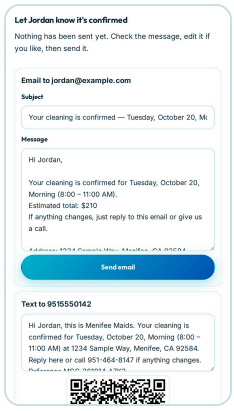


2.6 · The customer's invoice. The amount comes from the database, not the link, and **Pay** opens a Stripe checkout for exactly that amount.

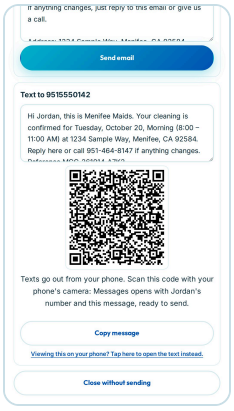
2 · PRODUCT TOUR

Messaging the customer, with a human in the loop

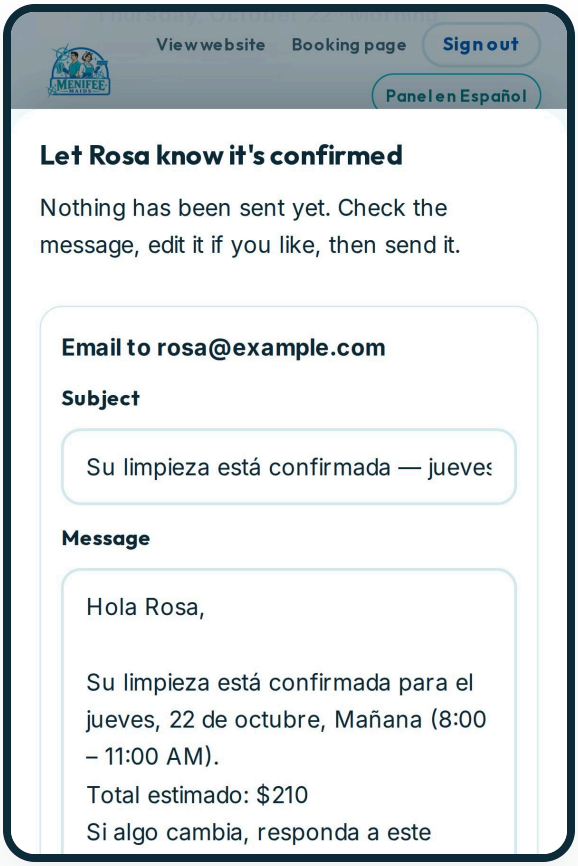
Every confirmation, change, proposal, cancellation and job-completion message is pre-written in the customer's language, but nothing goes out until a person reviews it and presses **Send**. Emails are sent by the platform from the business's own domain; texts are handed to the phone's messaging app.



2.9 · Review panel: editable email and text, nothing sent



2.10 · On a computer, a QR code hands the text to the



2.11 · On a phone, one tap opens Messages instead. Here the owner's English dashboard writes to a Spanish-speaking customer in Spanish.

> **Why this design**
>
> The client's owner asked for *control*: the ability to edit any message before it reaches a customer. I combined that with automation (pre-written, bilingual, sent from a verified business domain), so messaging is both fast and safe. A device-detection layer picks the right hand-off for the screen it's on, including phones running in "Desktop site" mode.

3 · VALUE DELIVERED

The operations hub that runs the business

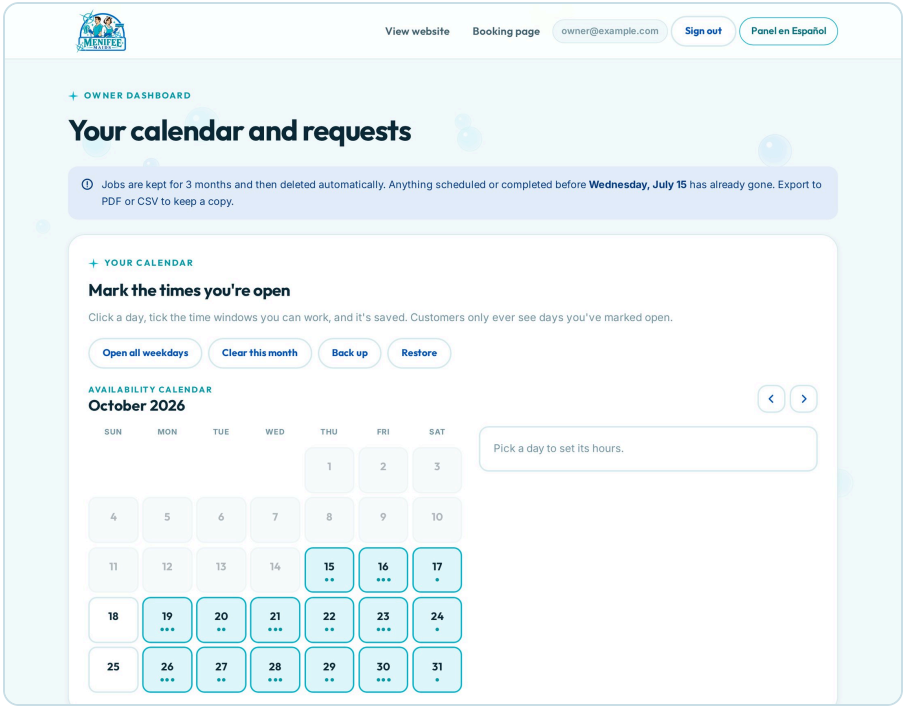
Behind the public site sits what changed the client's day-to-day: one place to see every request, run the schedule, manage every job and get paid, from any phone, in English or Spanish, for the owner and the helper alike.

BEFORE	AFTER	BUSINESS IMPACT
Requests arrived by phone call and text, and some were missed	Every booking is saved to the cloud database the moment it's made, and appears on every staff phone	No lost requests; every lead has a reference number and a full record

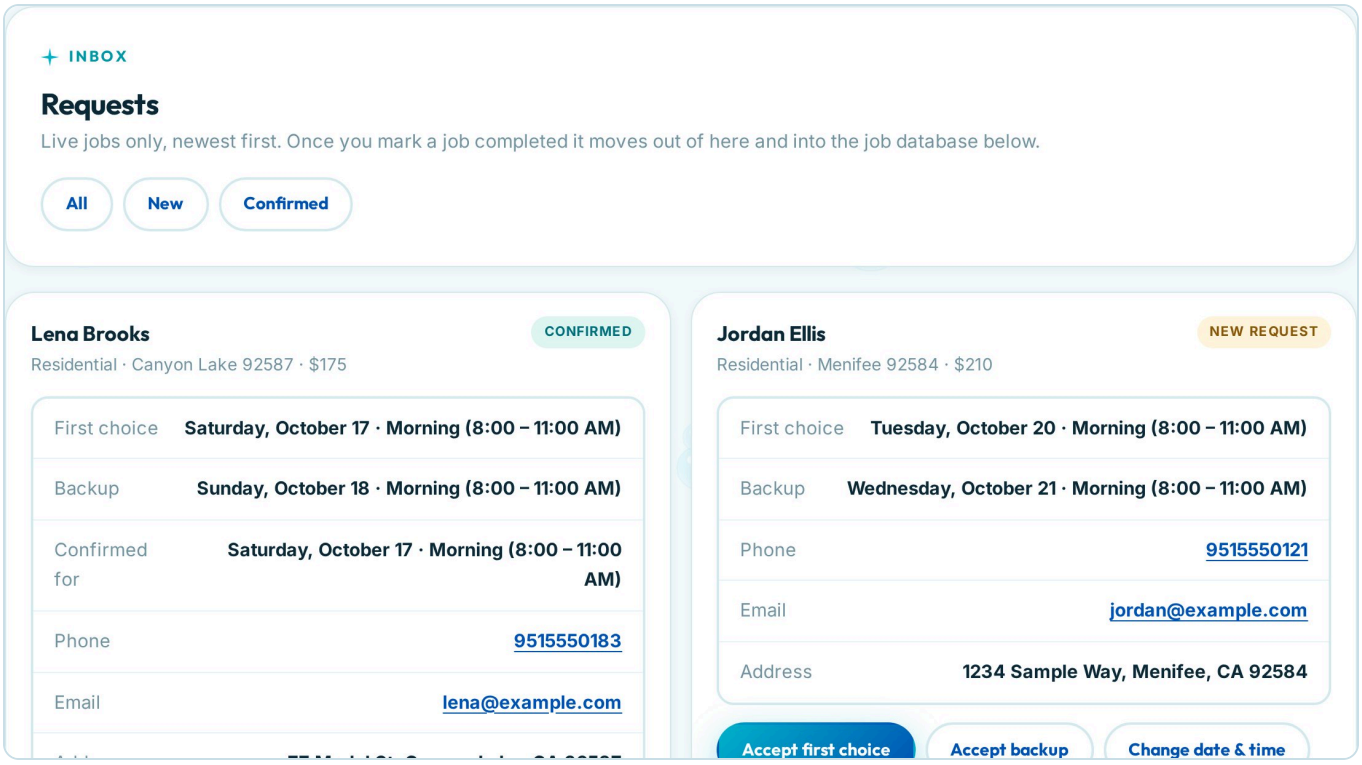
BEFORE	AFTER	BUSINESS IMPACT
Availability lived on a paper calendar	A shared calendar that staff edit from a phone, and that drives the booking page directly	Customers can only book times the business can actually work
Payments chased by hand, with no record of who had paid	Card payment links per invoice, marked Paid automatically, with live outstanding and collected totals	Faster collection and an always-current picture of revenue
Records scattered across texts and memory	A searchable job database with filters, exports to PDF and Excel, and a lifetime history per customer	Answers in seconds, clean records for taxes, insight into repeat customers
The owner did everything	A helper dashboard with exactly the access a helper needs, in English or Spanish	Work can be delegated without exposing business settings
English-only competitors	A fully bilingual site, dashboard and customer messages	Serves the large Spanish-speaking share of the local market

The owner dashboard

The owner's home screen combines the availability calendar with the request inbox. Tapping a day sets the time windows the business can work, and the booking page offers only those. New requests arrive with the customer's first choice, backup, contact details and access notes. Each can be accepted, rescheduled or cancelled in a tap, with the customer message pre-written for review.



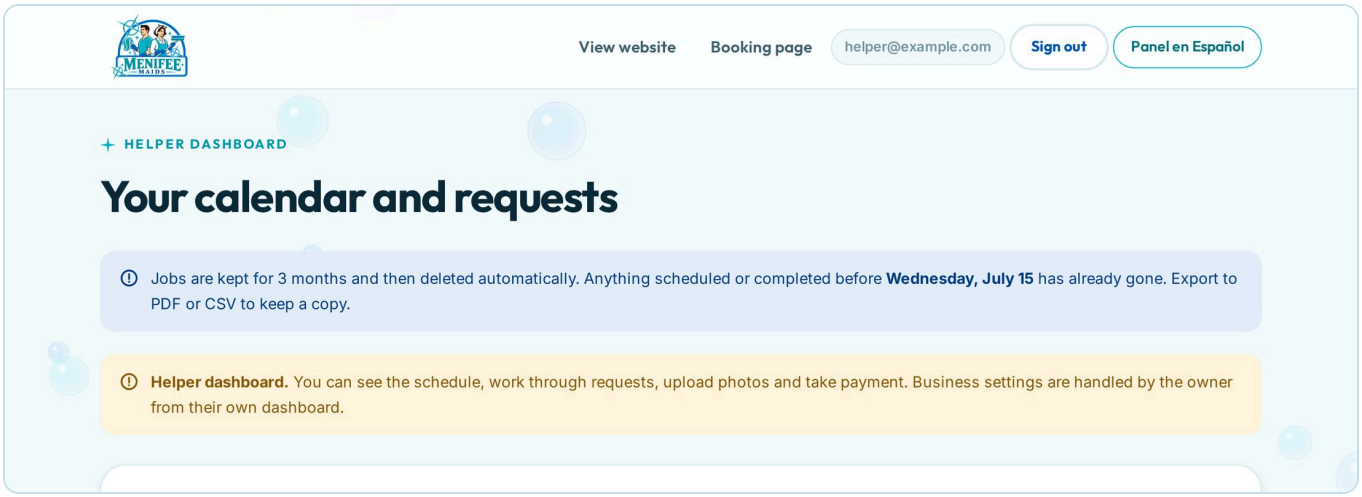
3.1 · The owner dashboard on a computer: the availability calendar with open days highlighted. Days marked open here are the only ones customers can book.



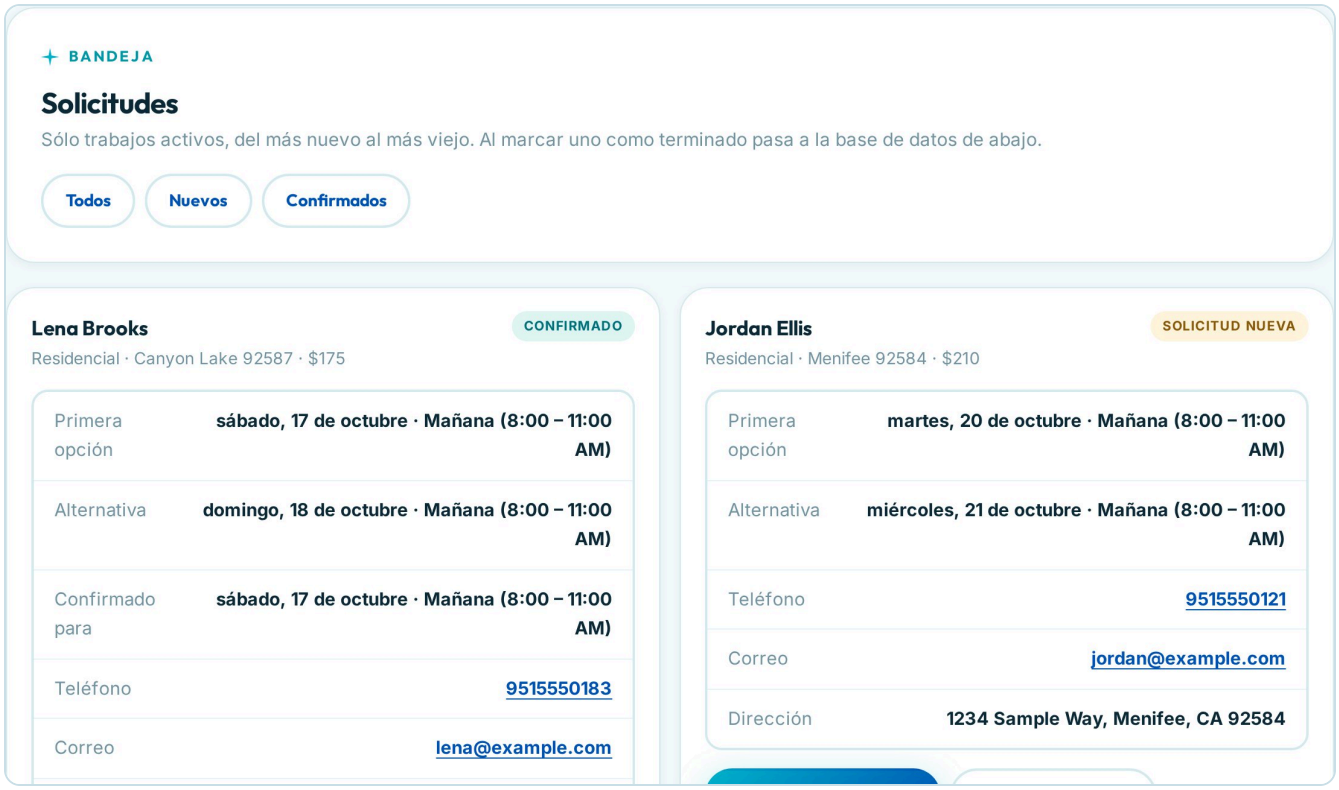
3.2 · The request inbox: each card shows the first choice, backup, confirmed time, phone, email and address, with one-tap actions to accept, change the date, send a payment link or mark the job complete.

The helper dashboards

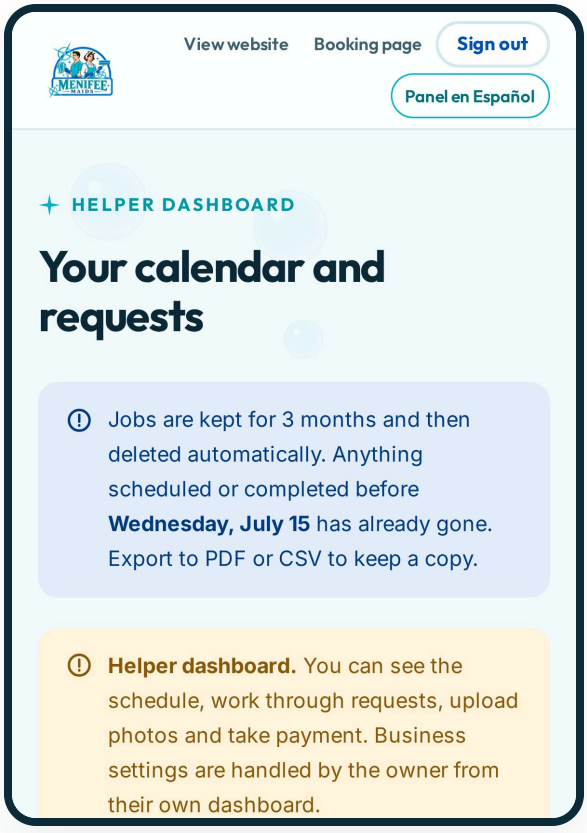
Helpers get the same fast tools with less power: they can see the schedule, work through requests, upload completion photos and send payment links, while business settings stay with the owner. The page explains this at the top, and the platform enforces it on every request. The whole dashboard is also available in Spanish, so the client can hire from the local workforce without a language barrier.



3.3 · The helper dashboard: a banner explains exactly what a helper can do, and business settings stay with the owner.



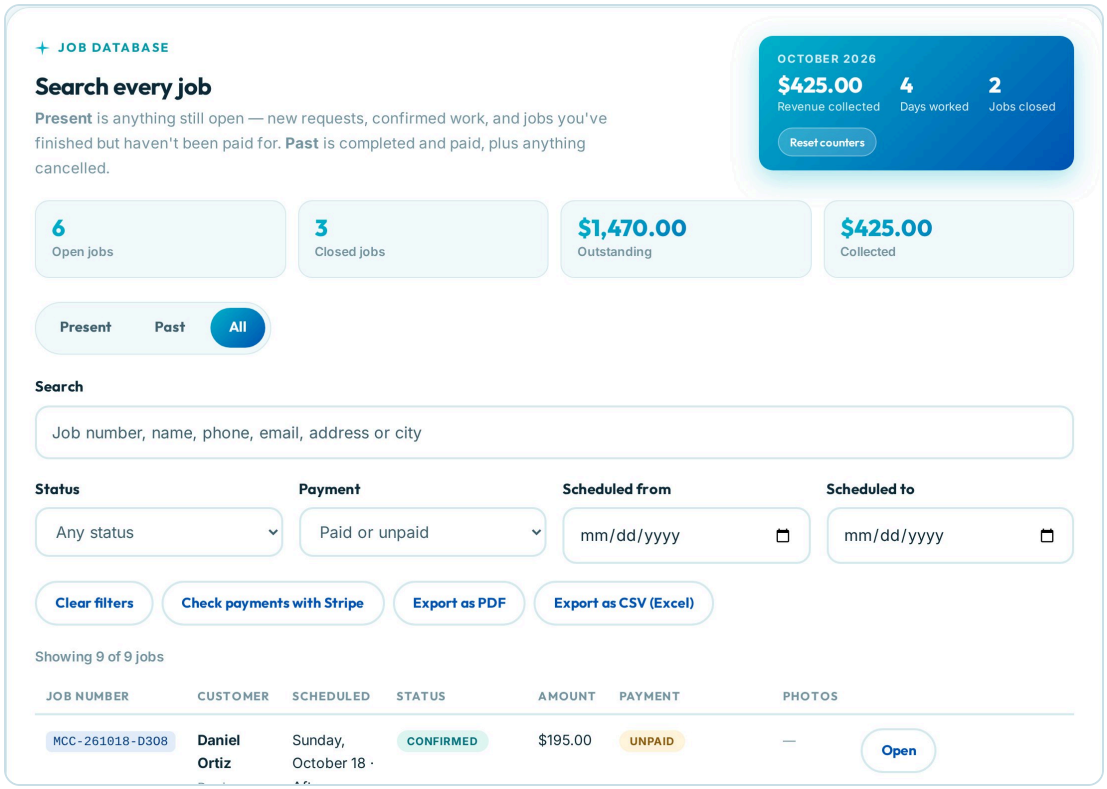
3.4 · The same request inbox in the Spanish helper dashboard: every label, status and date is in Spanish, while the data is shared with the owner in real time.



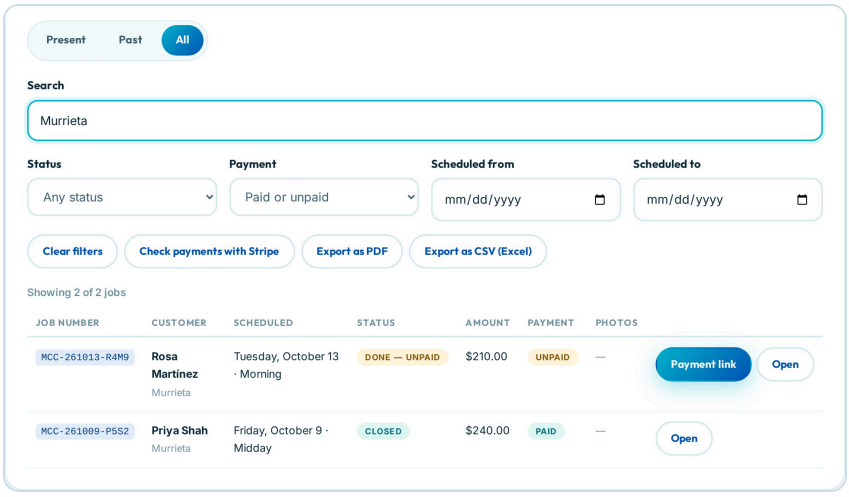
3.5 · On a helper's phone: installed to the home screen like an app, working on a weak signal and syncing in the

Managing and querying the job database

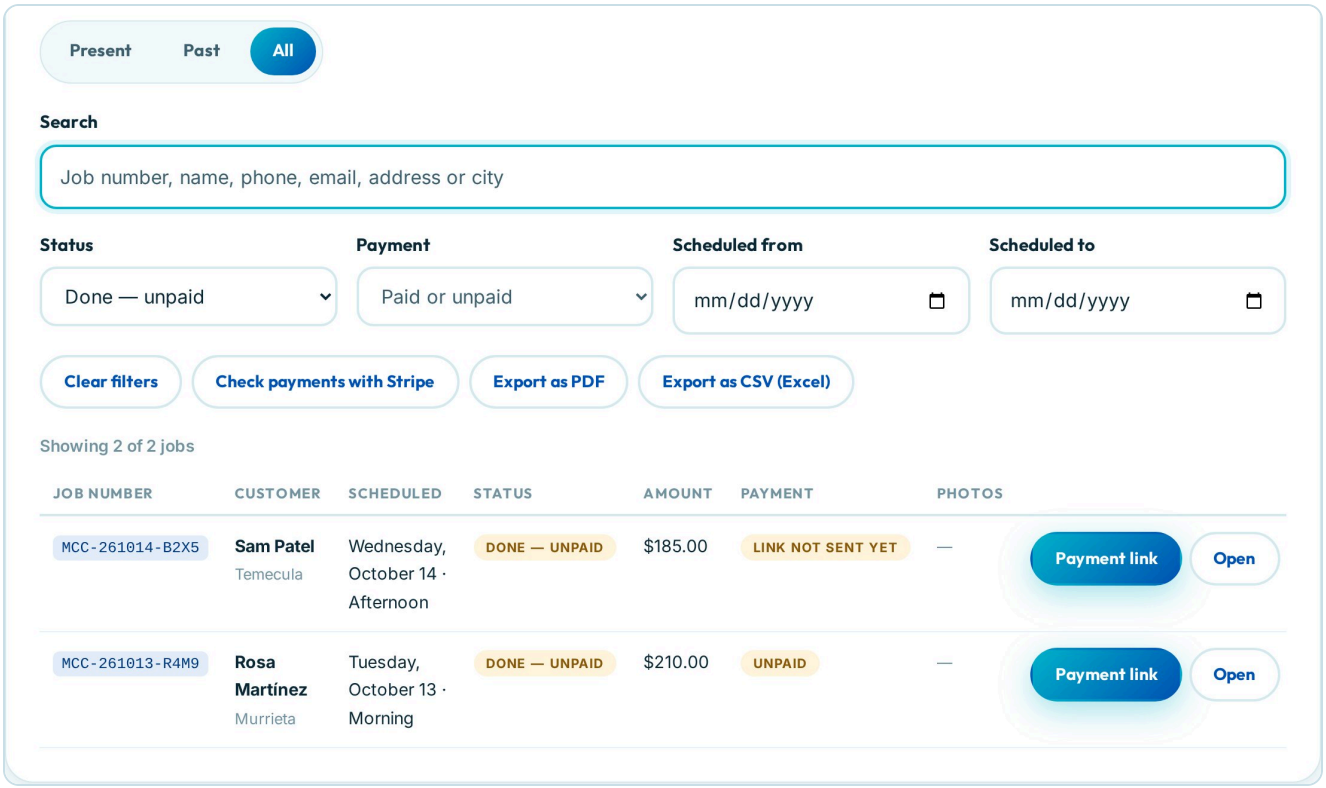
Every job, from first request to paid, lives in one database that staff manage from the dashboard. Live totals show open and closed jobs, and outstanding and collected revenue. Present, Past and All views separate active work from history. A single search box matches job numbers, names, phone numbers, email addresses, street addresses and cities, and filters narrow by status, payment and scheduled dates. Results export to PDF or Excel for records and taxes, and one button reconciles every payment against Stripe.



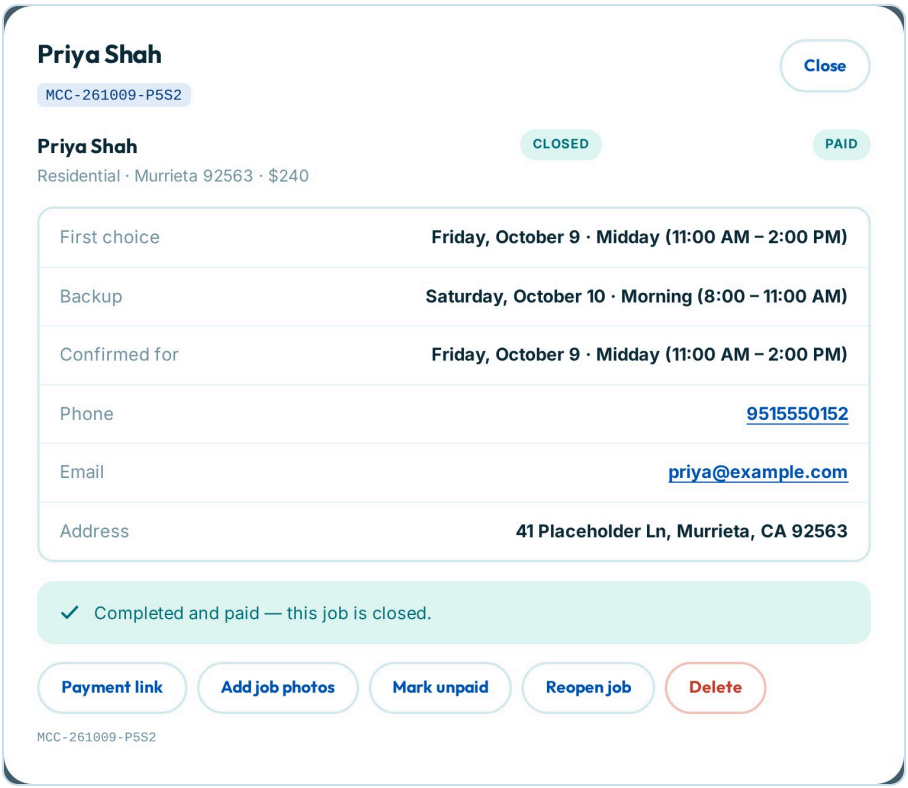
3.6 · The job database: live totals (6 open jobs, 3 closed, $1,470.00 outstanding, $425.00 collected), search and filters, export buttons, and every job with its status and payment state.



3.7 · A query by city: searching “Murrieta” instantly returns the two jobs there, one done but unpaid with a Payment link



3.8 · A query for money owed: filtering on status "Done — unpaid" lists every finished job awaiting payment, and shows which payment links haven't been sent yet.



3.9 · A job's full record: confirmed time, backup, contact details and address, with actions to send a payment link, add job photos, mark unpaid, reopen or delete.

Why it matters

The client went from reconstructing their business from text threads to answering “who still owes us?”, “what’s booked in Murrieta?” or “what did we collect this month?” in seconds, from a phone, in either language.

4 · ARCHITECTURE

A serverless platform on Azure

Customers and staff only ever talk to one front door: the Static Web App and its built-in API. Data, files, email and payments sit behind it; a separate Function App handles background work.



Figure 4.1 · Solid blue arrows: request path. Dashed blue: the queue is used only if a direct database write fails. Green: the Function App connects with its managed identity (no keys) and GitHub Actions deploys both apps. Violet: Stripe checkout and signed webhook calls.

Data model

CONTAINER	PARTITION KEY	WHAT IT HOLDS	RETENTION
`jobs`	`/id`	One document per job: query fields plus the dashboard's record	90 days after completion or cancellation

CONTAINER	PARTITION KEY	WHAT IT HOLDS	RETENTION
`payments`	`/jobId`	Invoice links: amount, token, expiry, paid status	Kept with the job's history
`availability`	`/monthKey`	Open days and time windows, one document per month	Rolling three months
`customers`	`/contactKey`	Lifetime summary: visits, spend, cities	Permanent (outlives jobs)

4 · ARCHITECTURE

Key design decisions

Each decision is recorded with its trade-off, the way I'd present an architecture decision record on a team.

DECISION	RATIONALE	TRADE-OFF & HOW I HANDLED IT
Static Web Apps (Free) with a managed API	One deploy for site and API, built-in Microsoft sign-in and role rules, zero hosting cost.	Managed functions have no managed identity, so the API uses keys held only in Azure environment variables, never in code or the browser.
Cosmos DB free tier, one container per entity	Schemaless JSON fits evolving job records; partition keys chosen per access pattern (`/id` , `/jobId` , `/monthKey`).	Closed jobs expire after 90 days to stay small and free; a monthly archive and nightly customer roll-up preserve history first.
Bookings write straight to the database, queue as fallback	Originally queue-first; I redesigned it so the customer path no longer depends on the background worker.	Durability preserved: if the write fails, the booking is queued and the owner is emailed if it ever exhausts retries.
Offline-first dashboard with cloud sync	The client's owner works from job sites with weak signal; the dashboard reads a local copy and syncs in the background.	Conflicts resolved with a three-way merge against a sync snapshot, plus a guard that restores data instead of mass-deleting it.
Server-side Stripe Checkout per invoice	Static payment links can't charge a unique amount; sessions are created from the amount stored in the database.	A tampered link can't change the price; payments reconcile automatically through a signature-verified webhook.
Human-approved customer messaging	The client wanted control over wording; messages are pre-written in the customer's language and editable.	One extra tap per message, accepted deliberately in exchange for control. Email is sent by the platform; texts are handed to the phone.

DECISION	RATIONALE	TRADE-OFF & HOW I HANDLED IT
Separate Function App with managed identity	Timers and queue triggers aren't available in managed functions; identity-based access means no secrets on that app.	Kept off the critical path, which proved valuable during its outage (section 8).

> **"Design the customer path so that no single background component can stop a booking."**

That principle came directly out of the Function App incident, and it's why the site kept taking bookings while that component was down.

5 · DELIVERY APPROACH

Phased delivery, verified at every step

The work ran in numbered phases, each verified before the next began. After launch, every change shipped as a versioned release through the same build, test, push and verify routine.

PHASE	SCOPE
1–4	Cloud foundation: Cosmos DB (free tier) with five containers and retention rules, Storage with private containers, queues and CORS, a Function App with managed identity, and Communication Services email on the client's domain
5–7	Source control, the Static Web Apps CI/CD pipeline, API health checks and local tooling
8–10	Application settings, background function deployment, custom domain, DNS and apex redirect
11–12	Staff invitations with owner and helper roles, two-step verification, and the dashboard installed on staff phones
13	End-to-end production testing, which led to the direct-to-database booking path and the cross-device sync engine
14	Stripe card payments, tested in sandbox and then switched to live with a verified real transaction
15	Automated SMS from a company number (planned; requires carrier registration)

How each phase was verified

1. **Platform health.** After every deployment, `/api/health` had to report both the API and the database as up.
2. **Data lands where it should.** Test bookings were checked in Cosmos DB Data Explorer, not just on screen.

1. **Access is enforced.** Each role was tested by signing in as owner and as helper, including a helper opening owner-only pages.

2. **Payments reconcile.** Stripe webhook deliveries had to return 200, and the job had to show Paid on every device.

3. **Releases are live.** Each release was confirmed by its cache version, sign-in, a booking, a payment and a customer message.

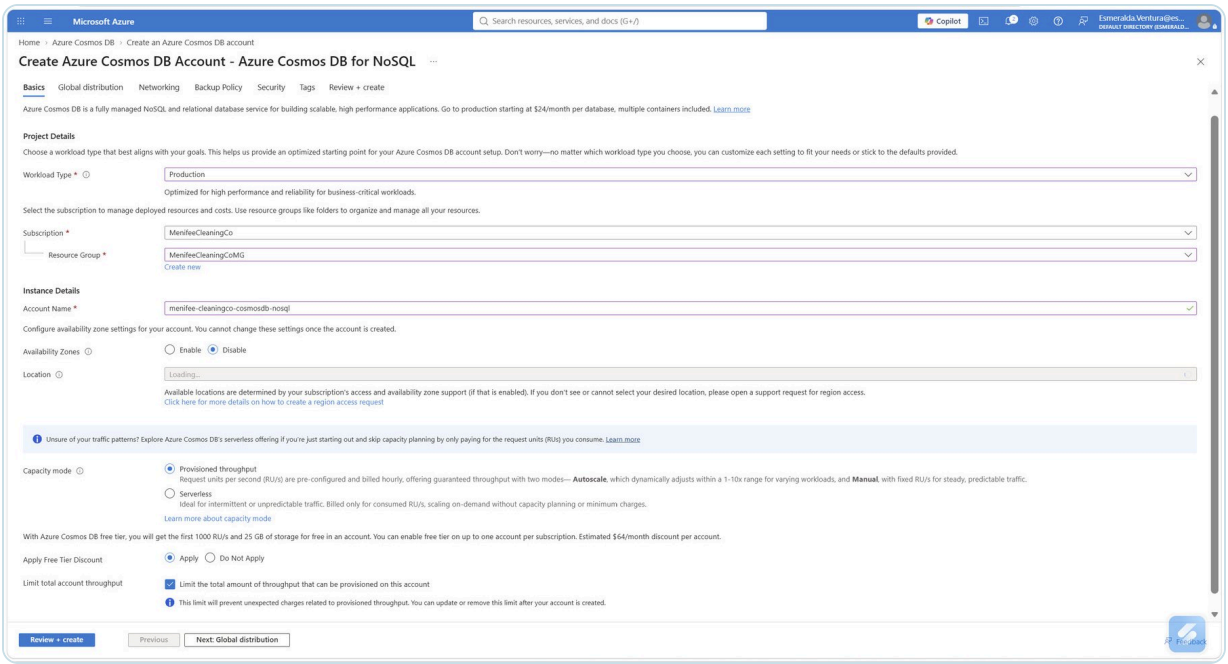
Technology stack

Front end HTML · CSS · JS · installable PWA	**API** SWA managed functions · Node 22	**Background** Azure Functions · timers · queues	**Data** Cosmos DB · Blob · Queue Storage
Messaging ACS Email · verified domain	**Payments** Stripe Checkout · signed webhooks	**Delivery** GitHub Actions · Azure CLI · PowerShell	**Domain** GoDaddy DNS · TLS · SPF/DKIM

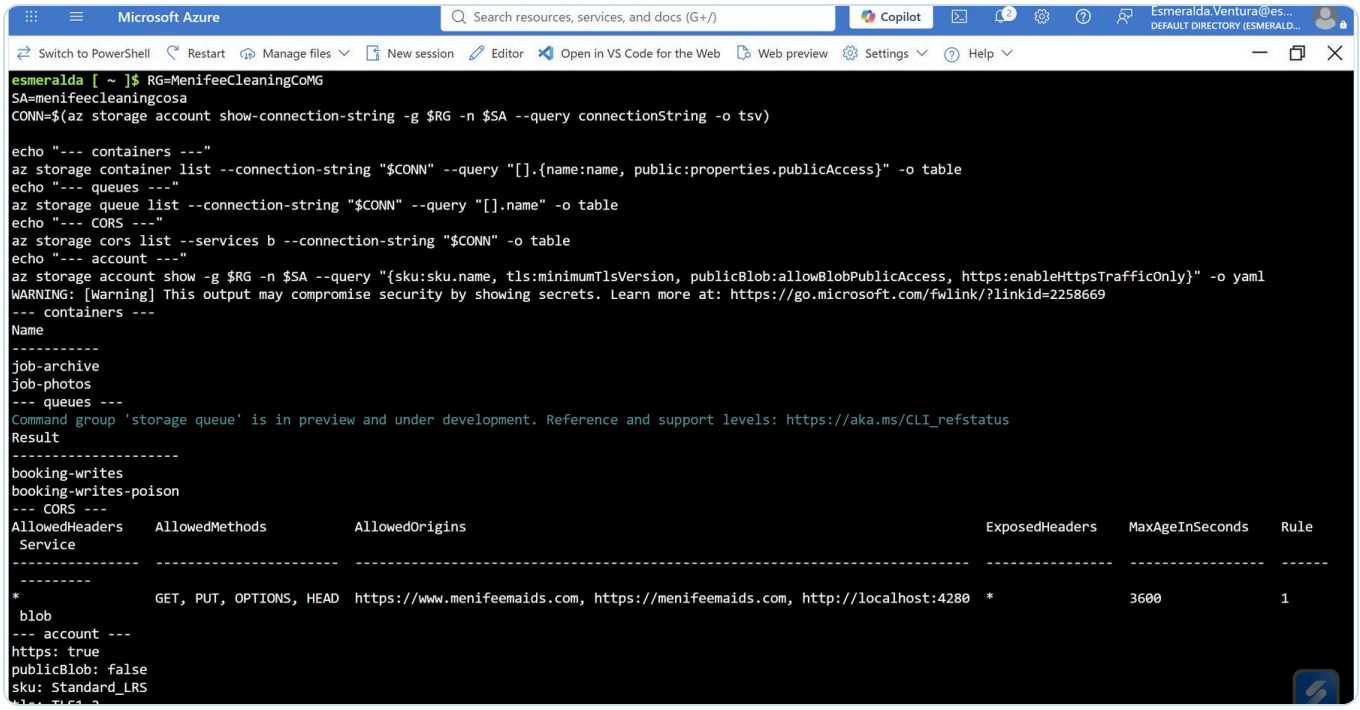
6 · BUILDING THE CLOUD FOUNDATION

Provisioning, step by verified step

I provisioned the environment through the Azure portal, Cloud Shell and the Azure CLI, verifying each phase before moving on and keeping a screenshot log of every step.



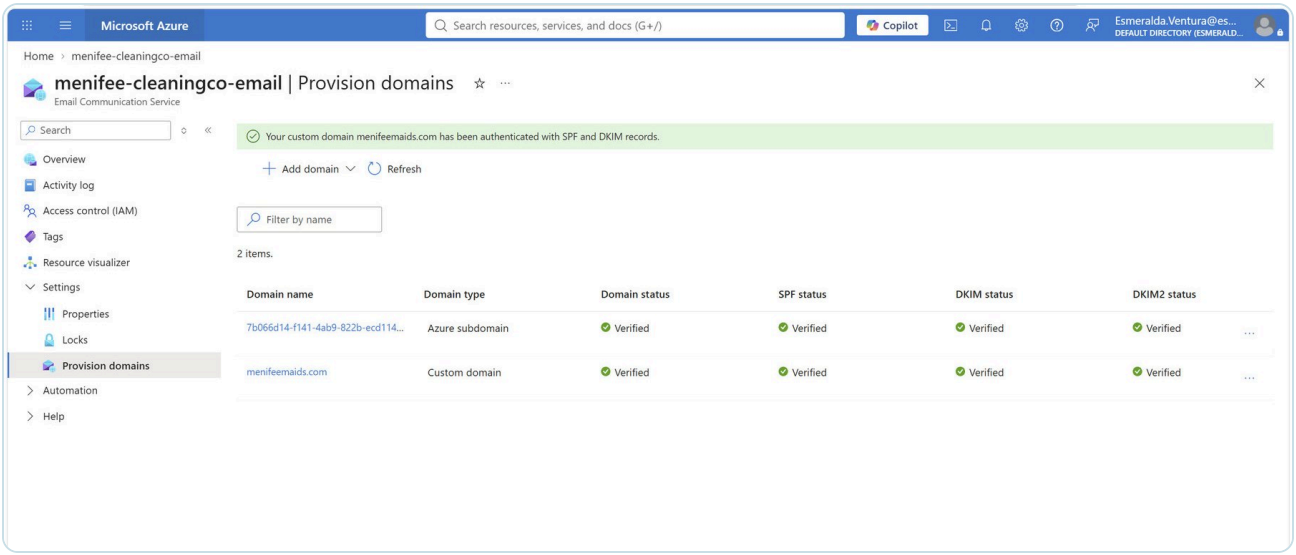
6.1 · Create Azure Cosmos DB Account (NoSQL), Basics tab: subscription MenifeeCleaningCo, resource group MenifeeCleaningCoMG, provisioned-throughput capacity mode, Apply Free Tier Discount set to Apply, and the option that limits total account throughput to stay inside the free allowance



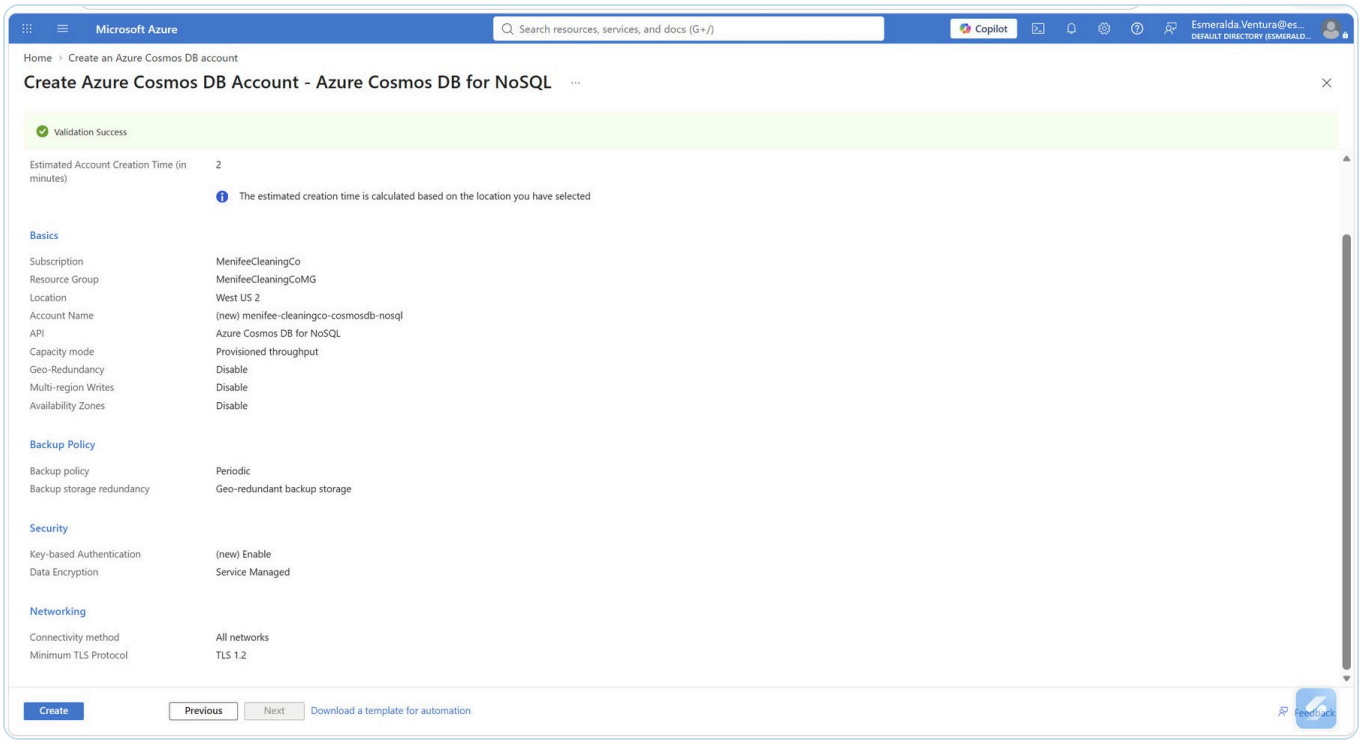
6.2 · Cloud Shell verification of the storage account: containers job-archive and job-photos, queues booking-writes and booking-writes-poison, a CORS rule limited to the site's domains (plus localhost for development), HTTPS only, public blob access off, Standard_LRS, TLS 1.2.

Key lines from this screenshot, transcribed for readability

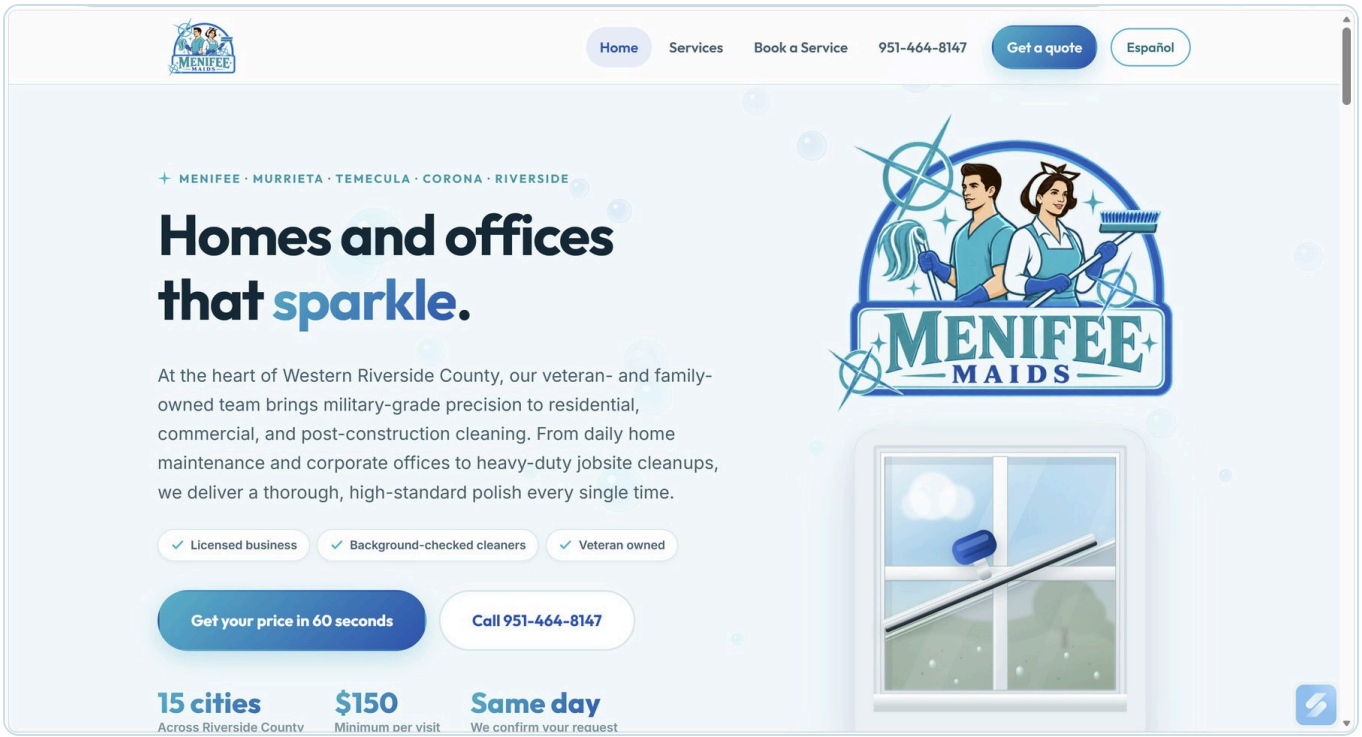
```
--- containers ---      job-archive, job-photos
--- queues ---          booking-writes, booking-writes-poison
--- CORS ---            GET, PUT, OPTIONS, HEAD
                        https://www.menifeemaids.com, https://menifeemaids.com,
http://localhost:4280
--- account ---         https: true   publicBlob: false   sku: Standard_LRS   tls: TLS1_2
```



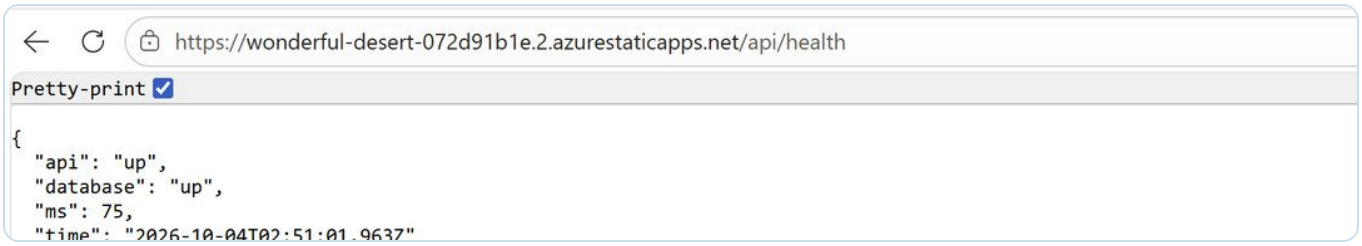
6.3 · Provision domains: the Azure-managed domain and menifeemaids.com both verified, with SPF, DKIM and DKIM2



6.4 · Review + create: validation passed. The summary shows West US 2, provisioned throughput, geo-redundancy and multi-region writes disabled, periodic backup, key-based authentication and minimum TLS 1.2.



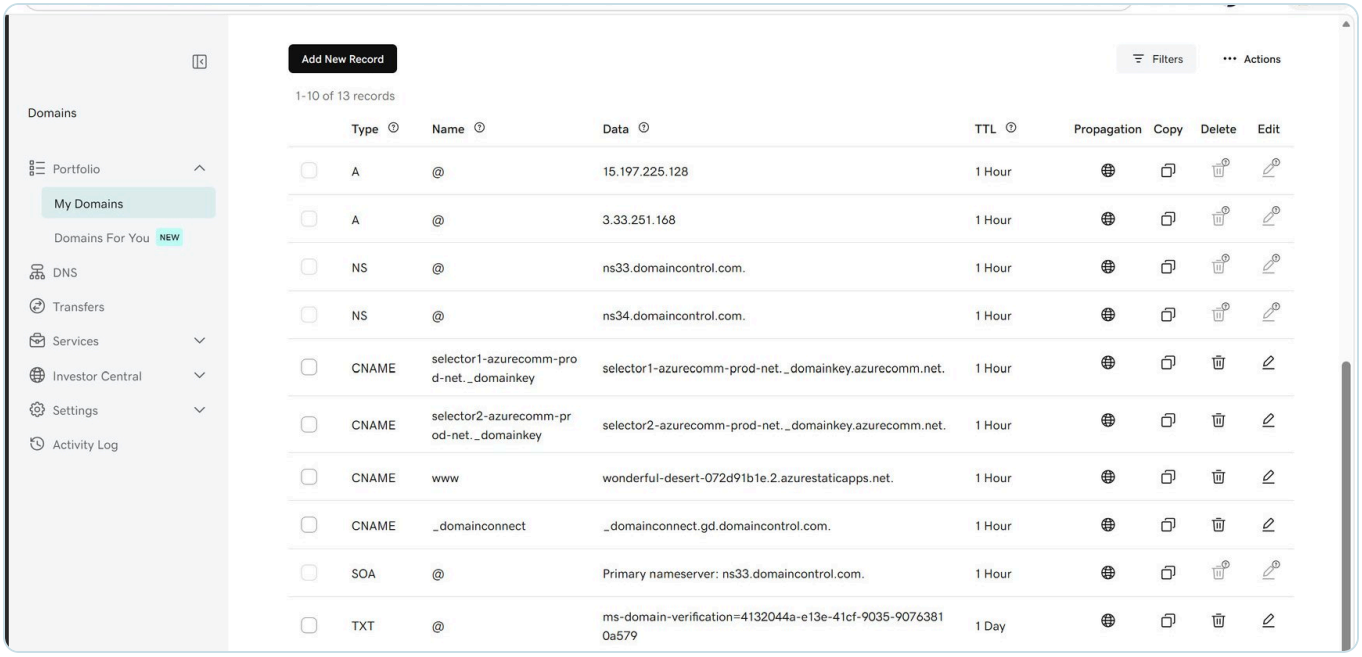
6.5 · The first successful deployment, served from the Static Web App's default azurestaticapps.net address.



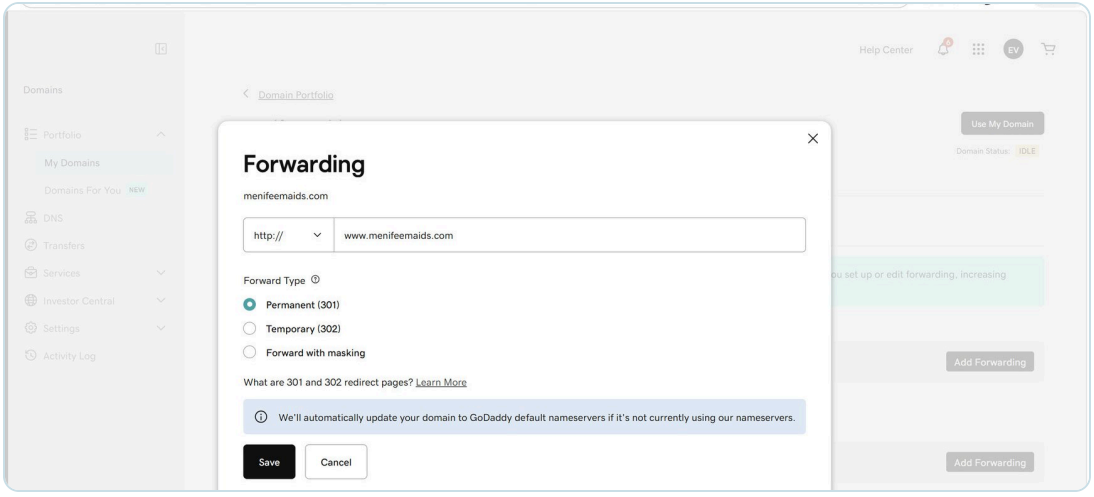
6.6 · The API health endpoint on the default domain reporting both the API and the database as up.

6 · BUILDING THE CLOUD FOUNDATION

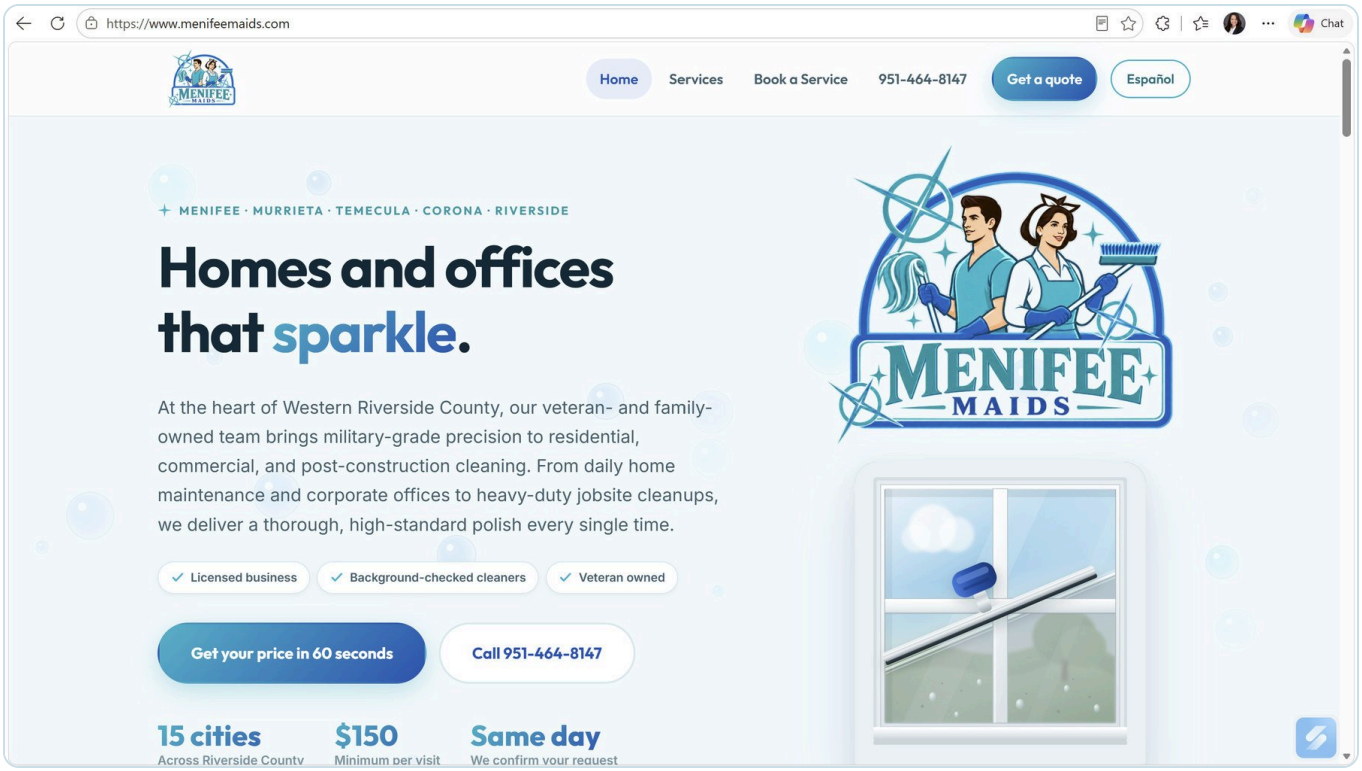
Custom domain, identity & access



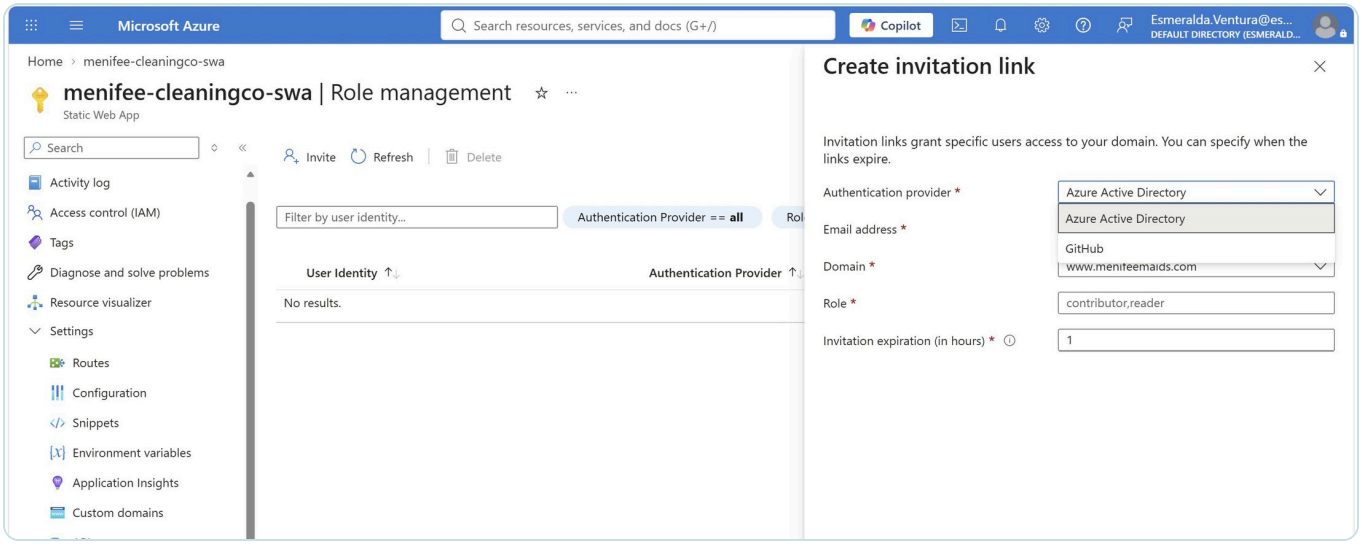
6.7 · GoDaddy DNS records for menifeemaids.com: the www CNAME for the site, and the records that verify and authenticate the email domain.



6.8 · The fix: a permanent (301) forward from the bare domain menifeemaids.com to www.menifeemaids.com, where



6.9 · The site live on www.menifeemaids.com.



6.10 · Role management on the Static Web App: creating an invitation link for a staff member with the Azure Active Directory provider, a role name and an expiry.

> **Access model**
>
> Route rules block helpers from the owner dashboard at the platform level, and every API function re-checks the caller's role from the signed principal header, so the check holds even if a page is reached directly. A helper who opens `/admin.html` is sent to an access-denied page. Both staff accounts use two-step verification.

A booking that can't get lost

The booking path validates the request, keeps the reference number the customer already saw, writes the job straight to Cosmos DB and sends both emails. If the database is unreachable, a durable queue takes over automatically.

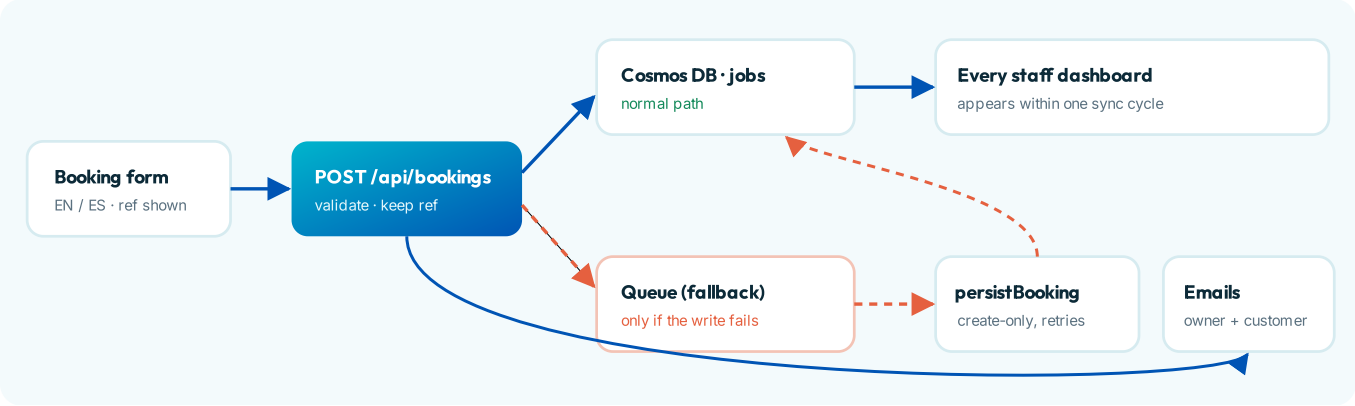


Figure 7.1 · Booking pipeline. The fallback writer only creates jobs that don't exist yet, so a delayed queued copy can never overwrite changes the owner has already made.

Keeping every phone in sync

Testing revealed that the original dashboard stored everything only in the browser, so a booking made on a customer's phone would never reach the owner's inbox. I rebuilt the data layer around an **offline-first sync engine** that keeps each device's working copy and Cosmos DB in step.

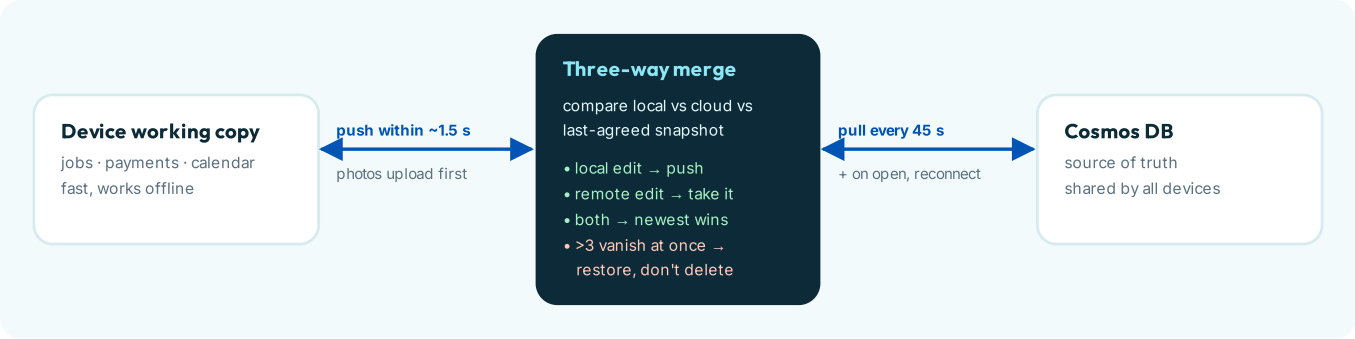


Figure 7.2 · Sync engine (`js/sync.js`). A device never pushes before its first successful pull, so a stale copy can't overwrite newer data. A status pill shows "Synced" or explains why not.

25/25
Sync checks passing: two staff phones and a customer, through create, confirm, pay, photo and delete

2 paths
Every booking has a fallback: if the database write fails, it's queued and written later

~1.5 s
From a change on one phone to it being saved to the cloud

Payments that can't be tampered with

The first design pasted a single Stripe payment link into settings, which can't charge each customer a different amount. I replaced it with per-invoice Stripe Checkout Sessions created on the server from the amount stored in the database, reconciled automatically by a signed webhook.

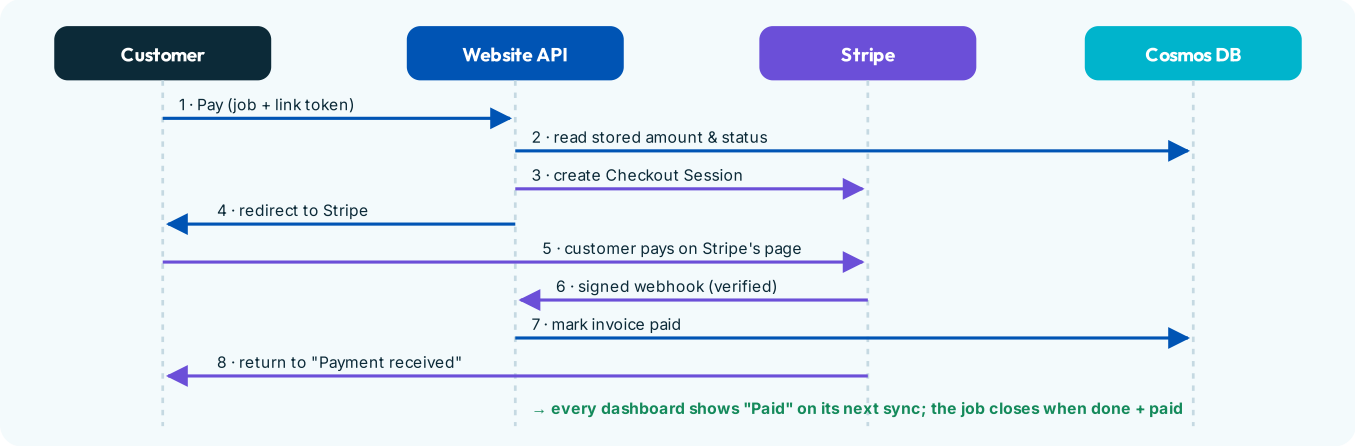
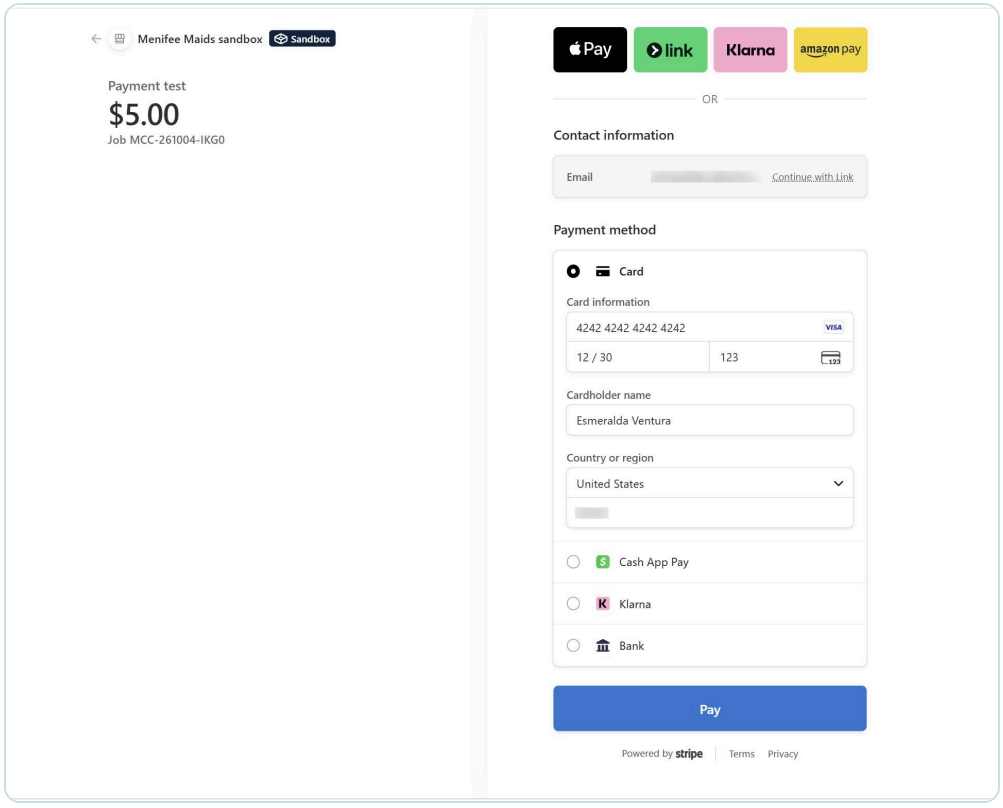
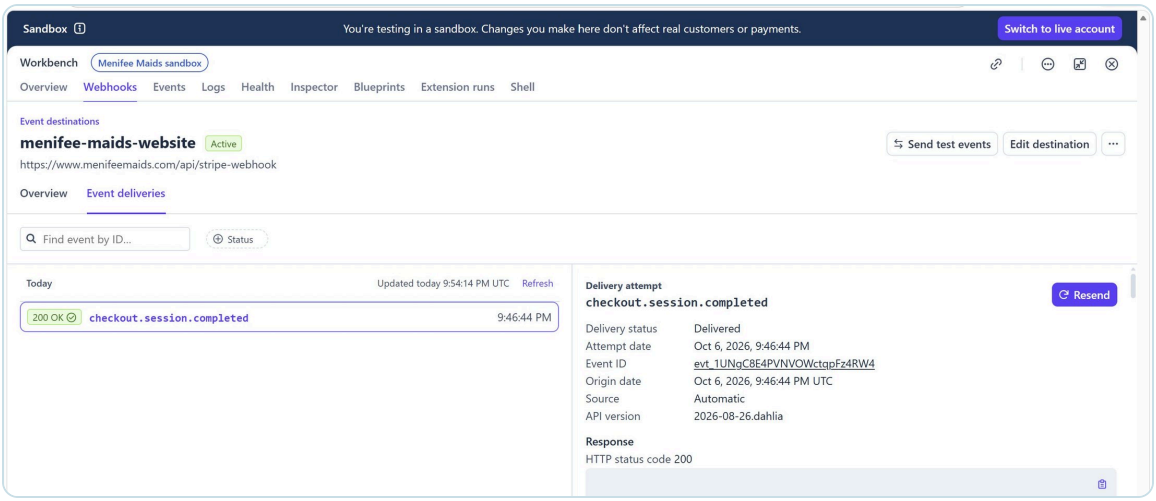


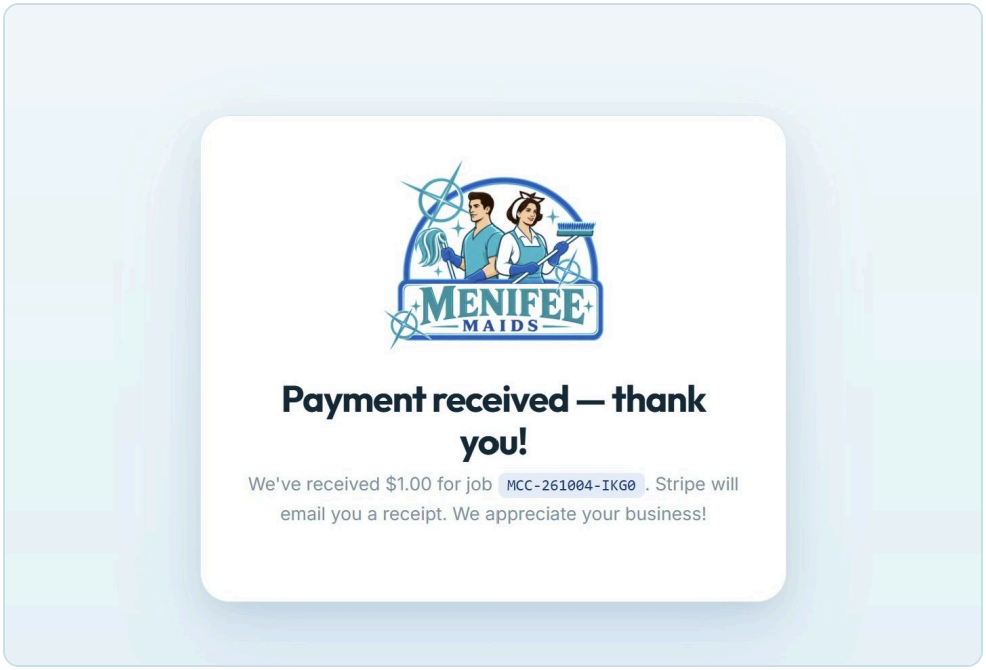
Figure 7.3 · Server-authoritative pricing: the browser never decides the amount. The Stripe key is restricted to "Checkout Sessions: write" and "PaymentIntents: read".



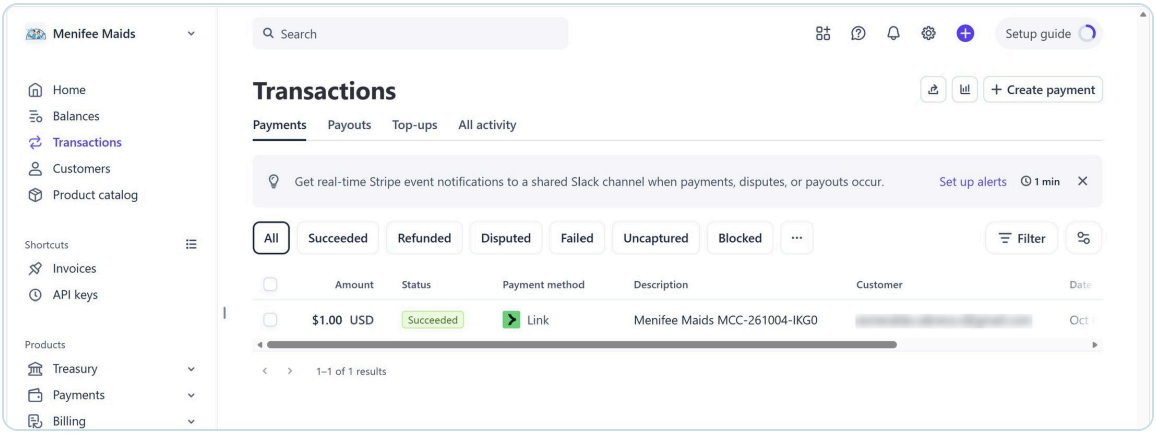
Stripe Checkout in sandbox for job MCC-261004-IKG0 ($5.00), opened from the website's invoice page and paid with Stripe's 4242 test card.



Webhook delivery log: checkout.session.completed delivered to the website and answered with HTTP 200.

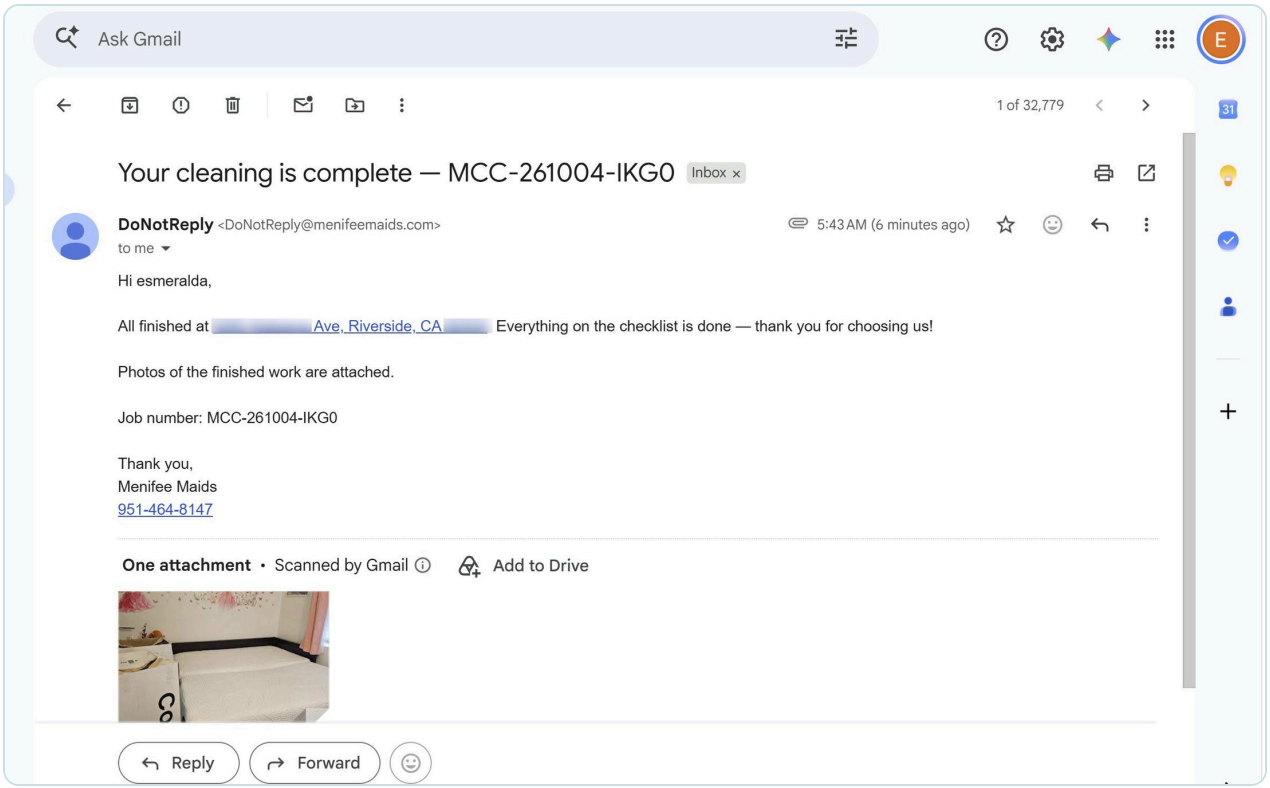


'Payment received — thank you!' after a real card payment in live mode (the session ID begins cs_live_).



Photos & messages customers actually receive

Browsers can't attach files to an email or text opened in another app, so I designed two delivery paths around that limit instead of fighting it.



7.4 · Delivered: the finished-job email from DoNotReply@menifeemaids.com, with the job photo attached.

1. **Capture.** Photos are resized to 1,400 px on the device and re-encoded, which also strips GPS location data from the files.
2. **Store.** The browser uploads straight to Blob Storage using a short-lived signed URL; the database keeps only the file path.
3. **Email.** The platform sends the email itself through Communication Services, with the photos attached (re-encoded to 1,280 px, capped at 7 photos and 8 MB).
4. **Text.** The message carries a private link to a photo page. The link's key is an HMAC of the job number, so it can't be guessed or edited to show another job.
5. **Hand-off.** On a phone, one tap opens Messages pre-filled. On a computer, a QR code (standard `SMSTO:` format) does the same through the phone's camera.

Incident: background Function App returned 503

My most demanding problem was a production component that never started. I kept the business running by redesigning around it, gathered evidence systematically, escalated to Microsoft and closed it out once the root cause was found.

Function App had zero workers

Resolved

SYMPTOM

Both the app and its deployment (SCM) endpoint returned `503 Service Unavailable`; the portal showed Runtime version "Error".

IMPACT

Background jobs (reminders, archive, customer summaries) couldn't run. Customer-facing features were unaffected after my redesign.

INVESTIGATION

Ruled out deployment method, network access, auth settings, app settings and package configuration; rebuilt the app from scratch and reproduced the fault.

ESCALATION

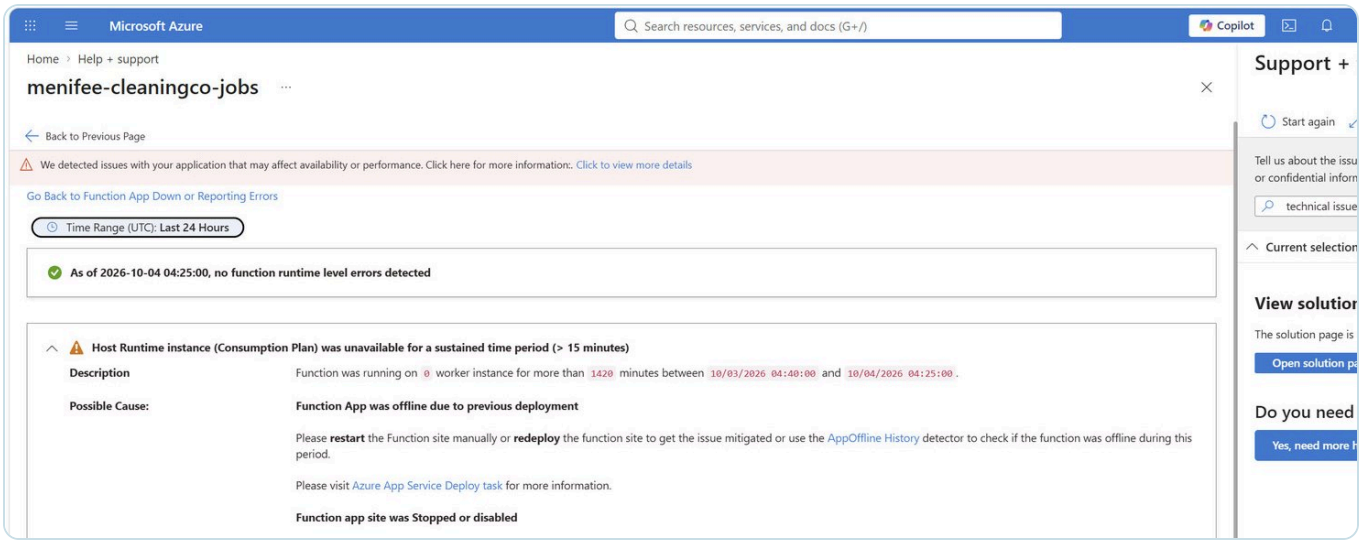
Filed an Azure support case with a full evidence trail: CLI output, diagnostics showing 0 running workers, and configuration screenshots.

ROOT CAUSE

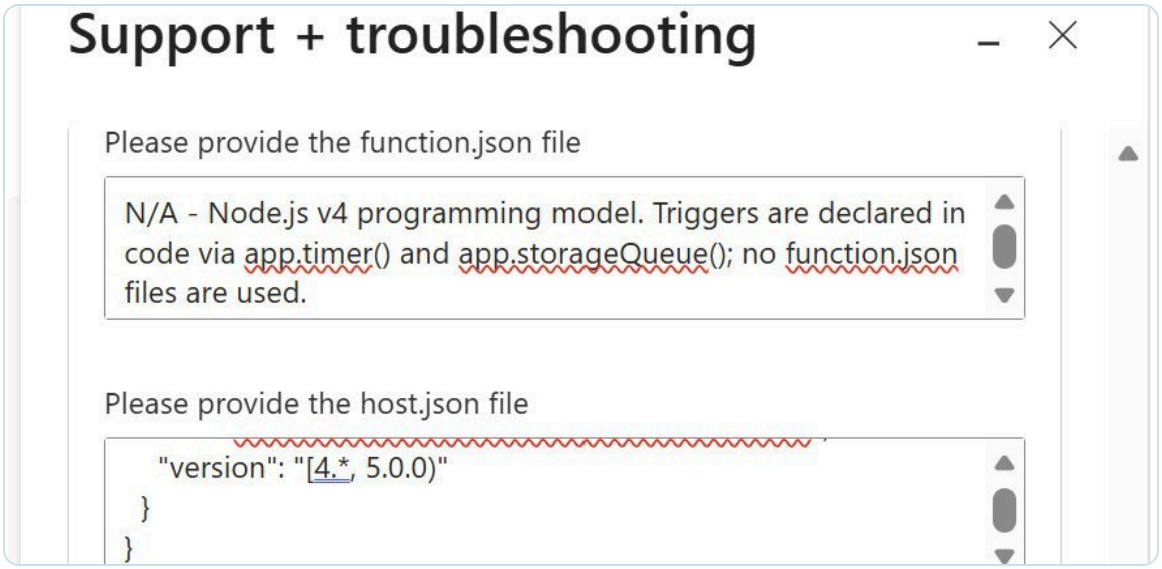
The app had been created on **Node.js 24**, which the Linux Consumption plan didn't support, so no worker could start. Microsoft Support confirmed the root cause.

FIX

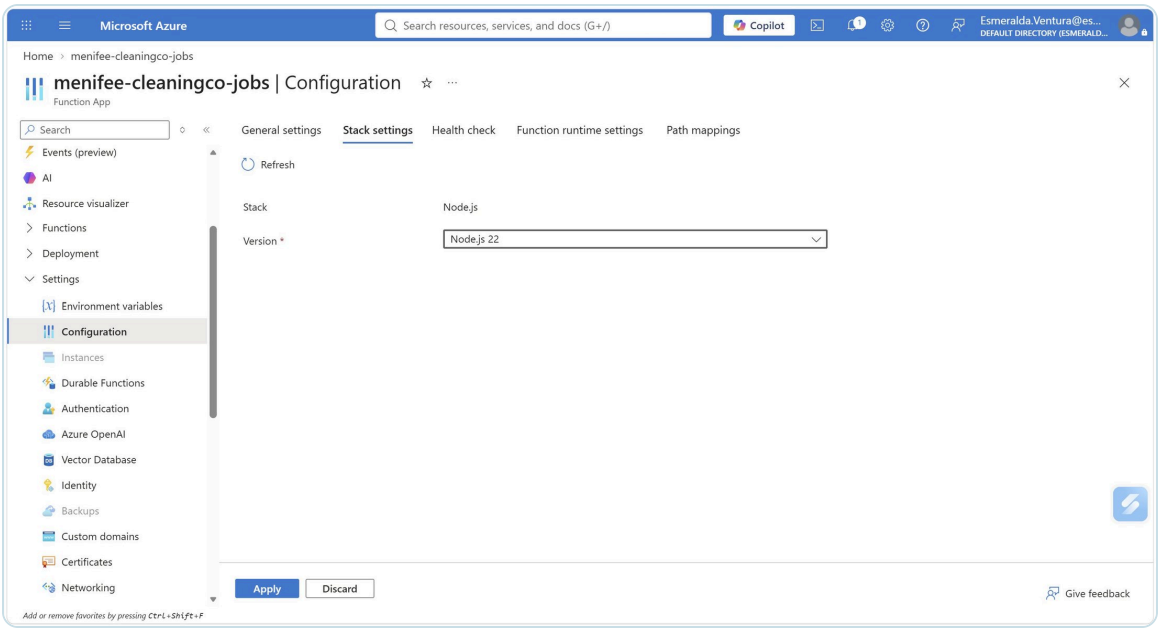
Switched the runtime to **Node.js 22** (runtime came up immediately), aligned the CI workflow's Node version, refreshed the deployment credential after a `401`, and redeployed: green on the first retry.



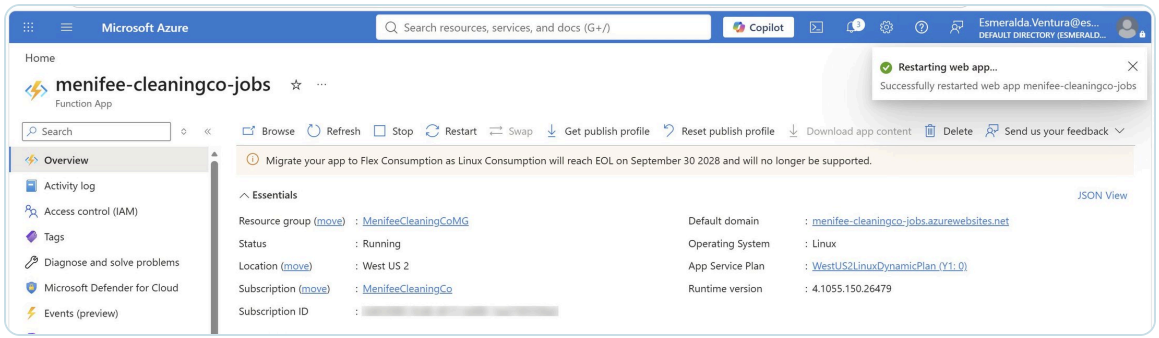
8.1 · Diagnose and solve problems: the host runtime instance (Consumption plan) was unavailable for a sustained period. The function app was running on 0 worker instances, so no code could execute.



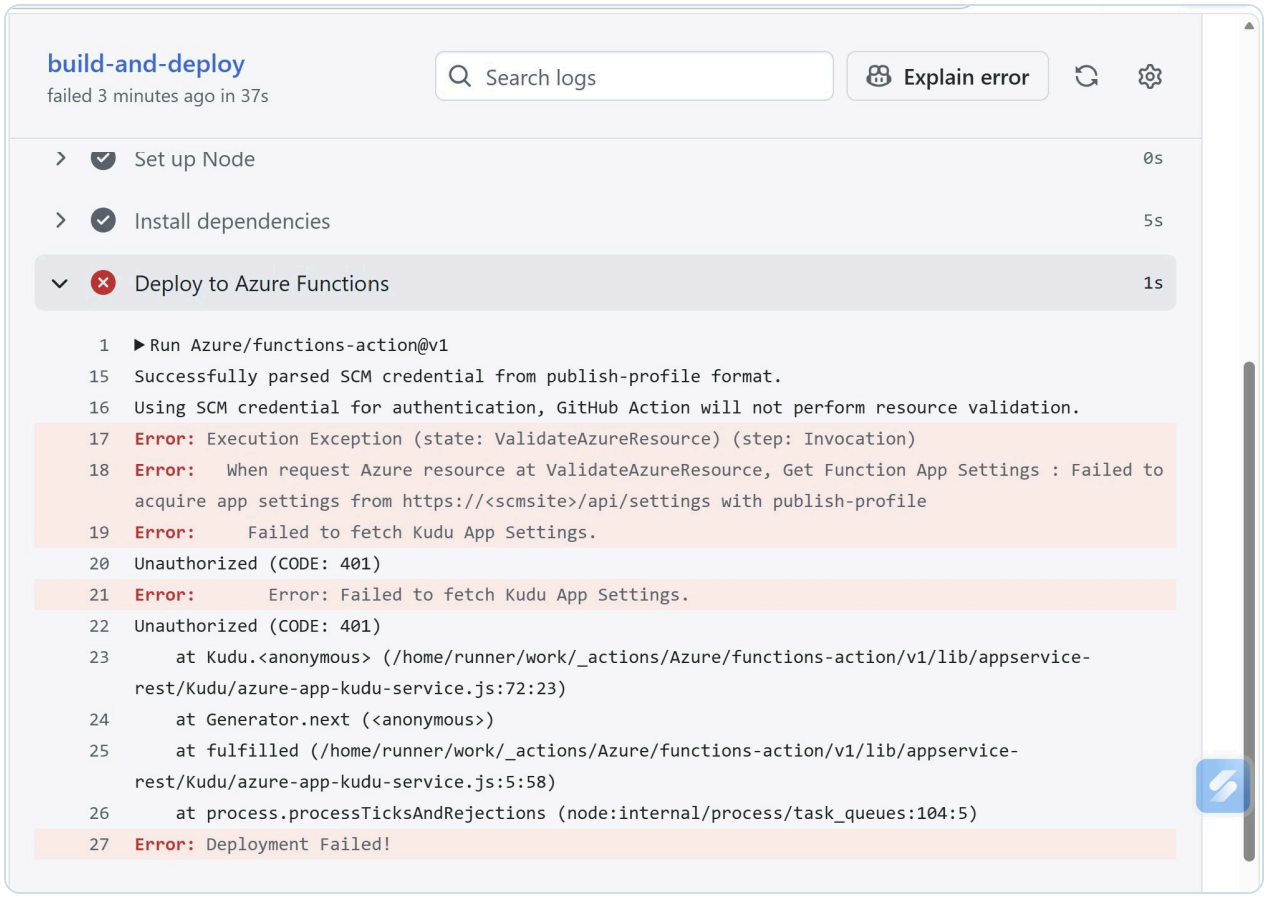
8.2 · Support case details: the function.json explanation (Node.js v4 programming model, with triggers declared in code, so no function.json files) and the host.json content.



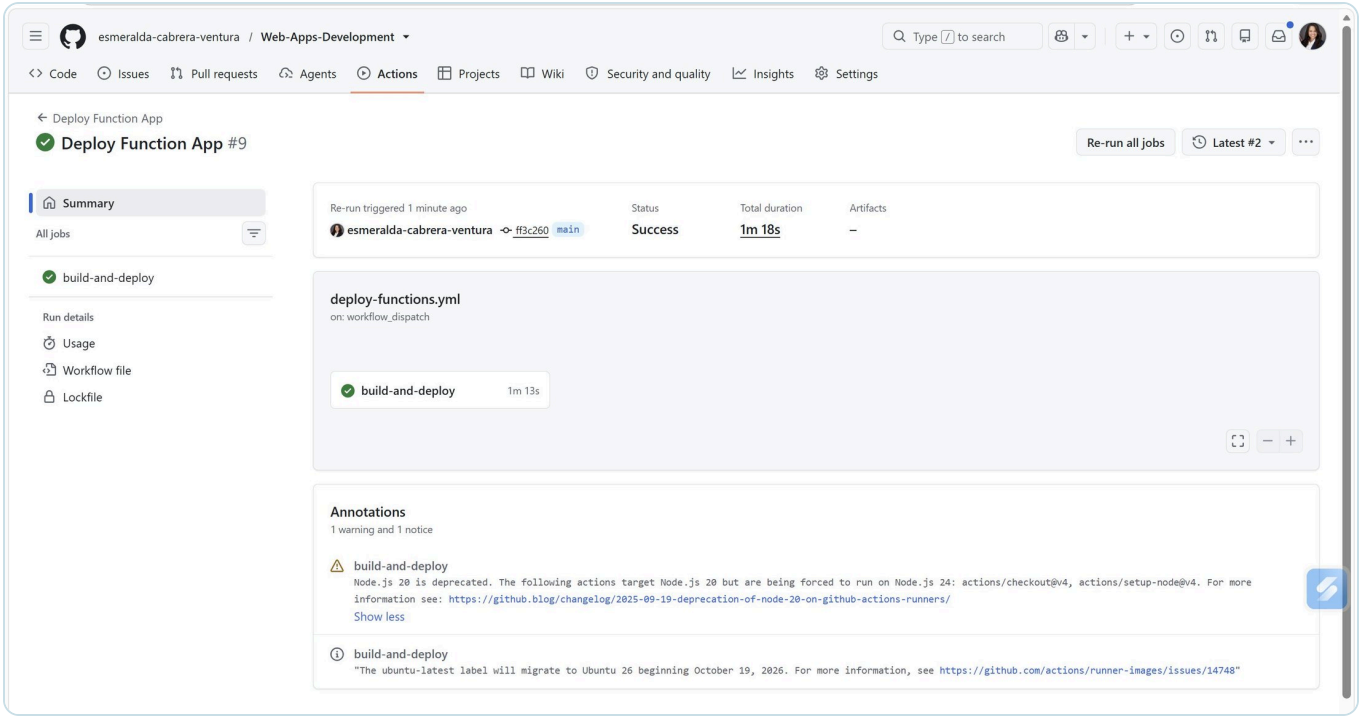
8.3 · Applying the fix from Microsoft Support: Configuration, Stack settings, Node.js version set to Node.js 22.



8.4 · 'Successfully restarted web app menifee-cleaningco-jobs': the app is Running with runtime version



8.5 · The remaining fault isolated in the deploy log: 'Failed to fetch Kudu App Settings — Unauthorized (401)', meaning the stored publish profile was no longer valid.



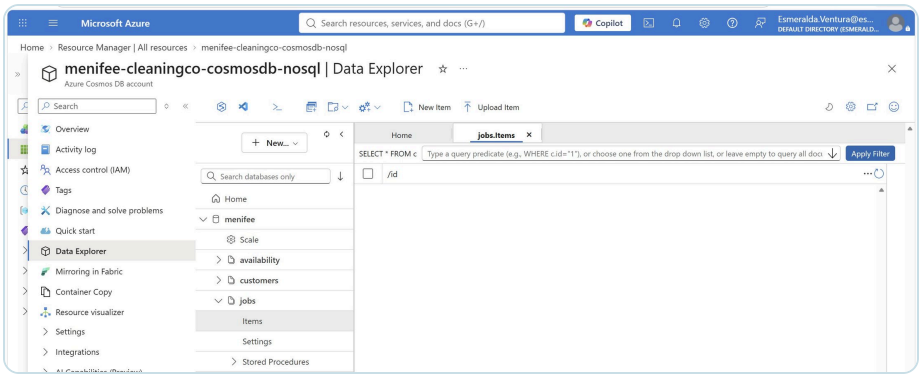
8.6 · Deploy Function App succeeds: build-and-deploy completes in 1 minute 13 seconds. The two annotations are

Finding the problems testing reveals

Bookings never reached the database

A test booking appeared in the dashboard, but Cosmos DB Data Explorer was empty. Rather than accept the apparent success, I traced it: the booking page and the dashboard shared the *same browser's* storage, so the "success" was local only. In real use, a customer's booking would never have reached the client's phone.

Resolution: direct database writes on the server, plus the cross-device sync engine (section 7), backed by a two-phone end-to-end test suite.

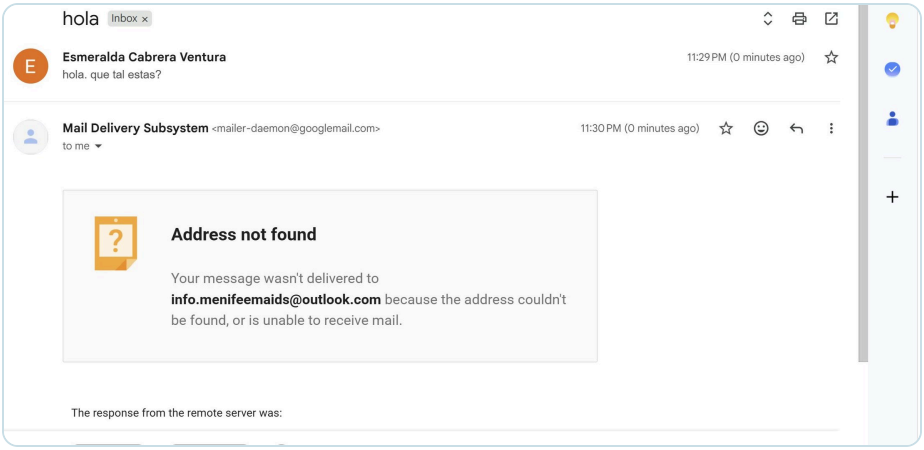


8.7 · Cosmos DB Data Explorer showing the jobs container empty after a test booking had appeared in the dashboard, which exposed that the original dashboard stored data only in the browser.

The business inbox didn't exist

The owner address used across the site couldn't be signed into, yet the name was "unavailable" for a new account. A single test email settled it: a hard bounce meant the mailbox had been closed, and **booking alerts and customer emails were silently going nowhere**.

Resolution: created a new verified mailbox, replaced the address in 31 places across 14 files, and updated the alert recipients, budget alerts and support-case contact.



8.8 · A single test email settles it: Gmail returns 'Address not found', so the mailbox no longer exists and booking alerts were going nowhere.

Smaller issues, same method

Every issue, large or small, went through the same loop: reproduce it, form a hypothesis, test one variable at a time, fix the cause rather than the symptom, then add a check so it can't come back silently.

Other issues found and fixed

ISSUE	ROOT CAUSE	FIX
Contact form text nearly invisible	Inputs inherited pale text color from a dark section	Explicit field colors; hidden heading restored
Logo squashed on sign-in pages	Width shrank via CSS while the HTML height attribute held	Global `img { height: auto }`; checked every page
Staff app opened the public home page	Sign-in redirect had no return address	`post_login_redirect_uri=/dashboard` on 401s
Texts failed from a computer	The share menu can't choose a contact; Phone Link rejects pre-filled messages	QR hand-off to the phone; safeguards for phones in "Desktop site" mode
Test cards "declined" in Stripe	Real cards are always declined in a sandbox	Documented test-card procedure; then verified live
Stripe account under another brand	An existing account's business name and icon would show to customers	Separate Menifee Maids account with its own branding and payouts
Sideways scroll on small phones	Long email addresses and a fixed-width card	Wrapping rules; verified 13 pages × 7 screen widths
Past time slots still offered	The date was checked but not the time of day, using the visitor's own time zone	Pacific-time filtering with a one-hour lead, enforced by the API as well
English left in the Spanish helper portal	A sentence split around a link, and a template table used for the wrong text	Missing translations added; a full scan of every portal state comes back clean
Helpers struggled to send payment links	The button was buried, and links went out from personal apps	Review-panel sending for owner and helper, plus a Payment link button in the job database

What I'd highlight to a hiring team

I didn't let a blocked dependency block the business. While the case was open, I moved bookings off that component entirely (direct database writes with a queue fallback), so customers never noticed. When the root cause arrived, I verified the fix layer by layer (runtime, then CI, then credentials) instead of retrying blindly.

9 · TESTING & QUALITY

Proving it works before customers do

For every release I ran the **real API code** against in-memory stand-ins for Cosmos DB, Blob Storage, the queue, email and Stripe, then drove the **real pages** in a headless browser as customers, owners and helpers on simulated phones and computers.

Automated end-to-end checks by suite



Total: **130 checks**, plus layout verification of 13 pages at 7 screen widths (320 px to 1,280 px) and both booking forms in English and Spanish.

What the suites verify

1. Two staff phones and a customer stay consistent through create, confirm, reschedule, pay, photo and delete.
2. Real QR images are decoded to confirm the number and message are exact.
3. Forged webhooks are rejected; edited payment links can't change the charge.
4. Nothing is ever sent to a customer without a Send press.
5. A wiped device restores from the cloud instead of deleting shared data.

Quality habits

Regression suites were rerun on every version, v4 through v14. When a test failed because of the test itself (a placeholder assertion, or a size threshold too strict for synthetic images), I fixed the test and said so, rather than loosening the product. Visual changes were checked with rendered screenshots at phone and desktop sizes.

10 · DEPLOYMENT & DEVOPS

Repeatable releases through GitHub Actions

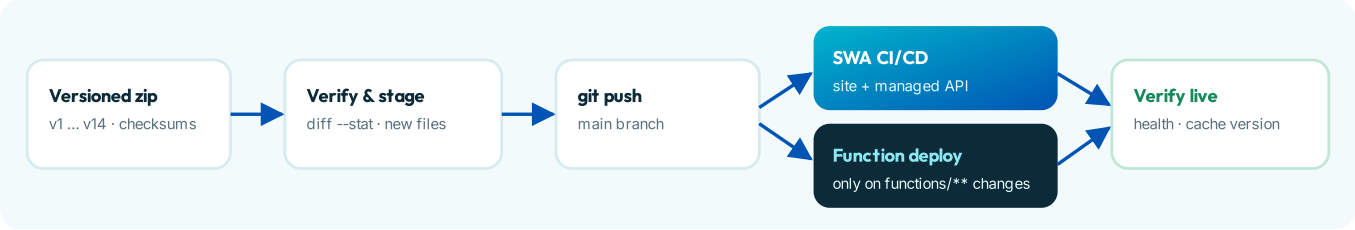
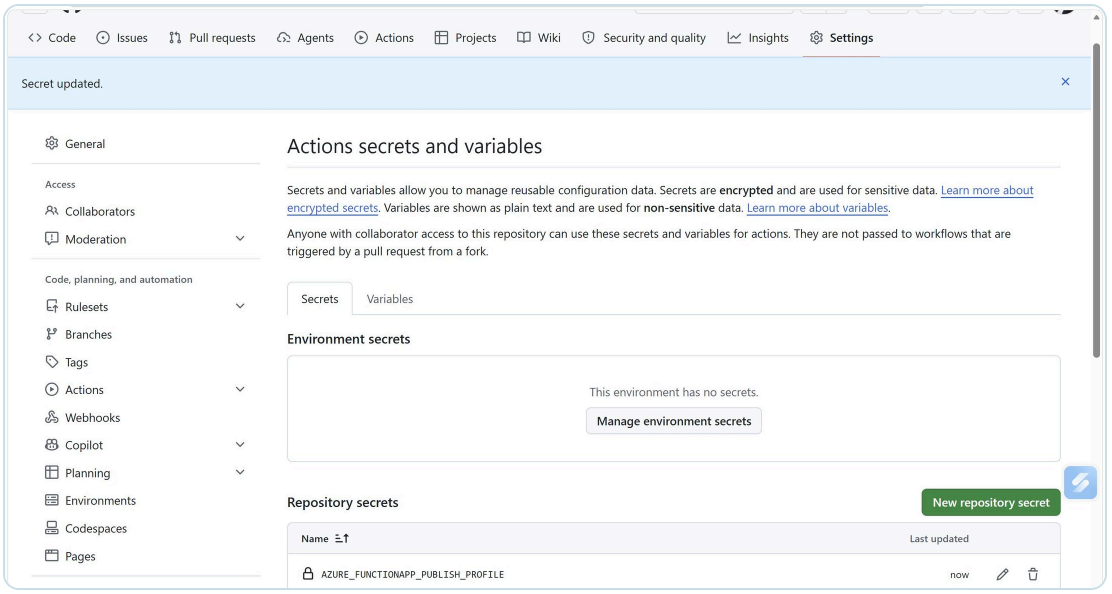


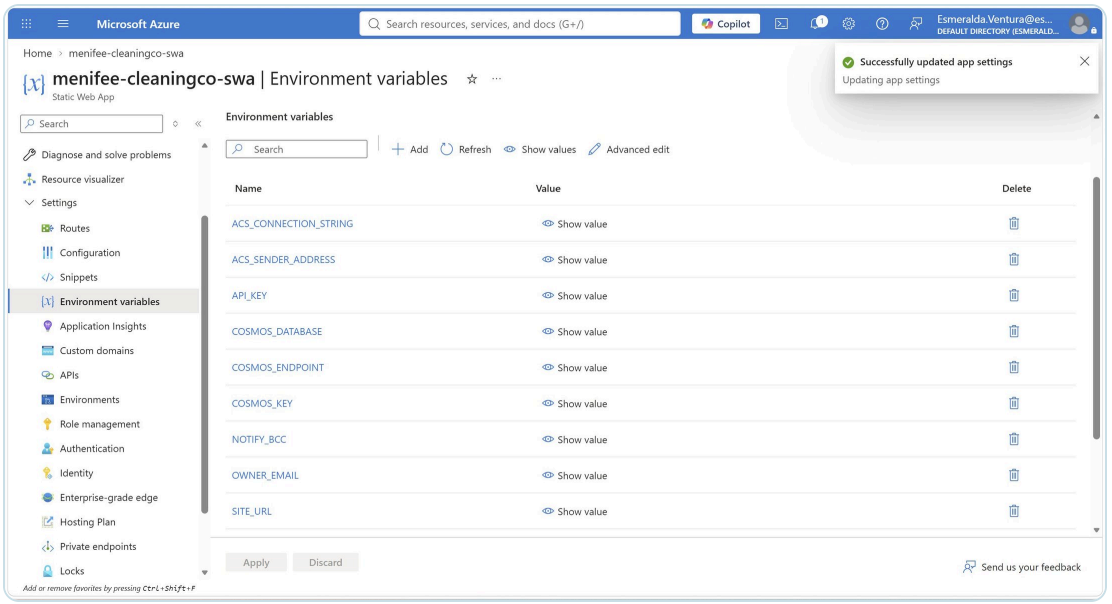
Figure 10.1 · Release flow. Each release bumps the service-worker cache version so phones pick up new code, and each push is checked for untracked new files, a silent-failure mode I documented and guarded against.

Release history

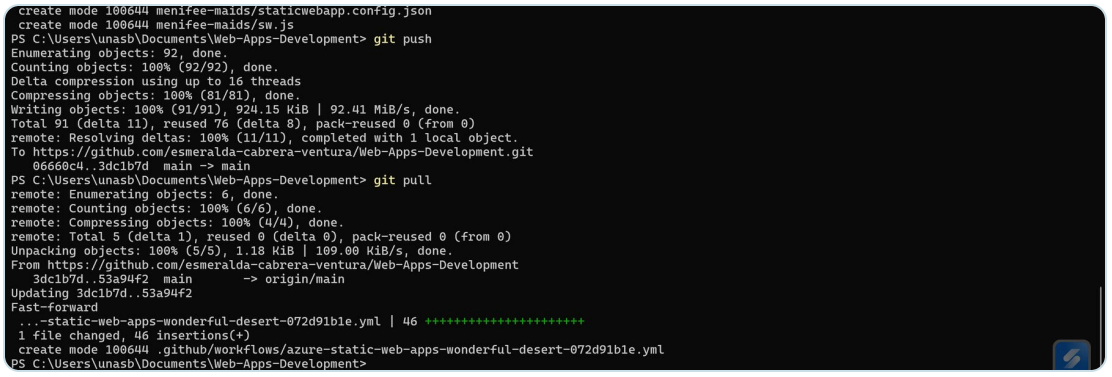
VER.	HIGHLIGHTS
v1–v3	Readable contact form; replacement business mailbox across the site; image aspect-ratio fix on all pages; README rewritten for a professional audience
v4	Direct-to-database bookings with queue fallback; cross-device cloud sync; real calendar on the booking page; one reference number everywhere
v5	Per-invoice Stripe Checkout, signature-verified webhook, simplified settings
v6–v8	Bilingual customer notifications, Cancel job, and human approval before any message is sent
v9–v10	Full menu on phones; duplicate content removed; Spanish copy fixed; photos by email attachment and private link
v11–v13	QR hand-off for texts; phone-detection safeguards; sign-in returns staff to the dashboard
v14	Owner and helper send payment links through the review panel, with a Payment link button in the job database and a "Link not sent yet" status; the booking calendar hides past and imminent times (Pacific time, enforced by the API); Spanish helper portal fully translated



'Secret updated.' in the repository's Actions secrets.



'Successfully updated app settings' after applying the live Stripe key and webhook secret.



Pushing the application to GitHub (92 objects), then pulling back the GitHub Actions workflow that Azure Static Web

Security built in, not bolted on

Identity & access

Microsoft sign-in with invitation-only roles; route rules plus per-function role checks; two-step verification on staff accounts; helpers can't reach owner settings.

Secrets

Keys live only in environment variables and GitHub encrypted secrets. The Stripe key is restricted to two permissions. The Function App uses a managed identity with no keys at all. Every credential is scoped to a single purpose and can be replaced without touching code.

Payments

Card data never touches the site (Stripe Checkout). Amounts come from the database; webhooks are signature-verified; unguessable link tokens; links expire and can be replaced.

Customer data

Photos stripped of location metadata; private photo links protected by HMAC keys; signed, time-limited storage URLs; the cloud API is never cached by the service worker.

Messaging compliance

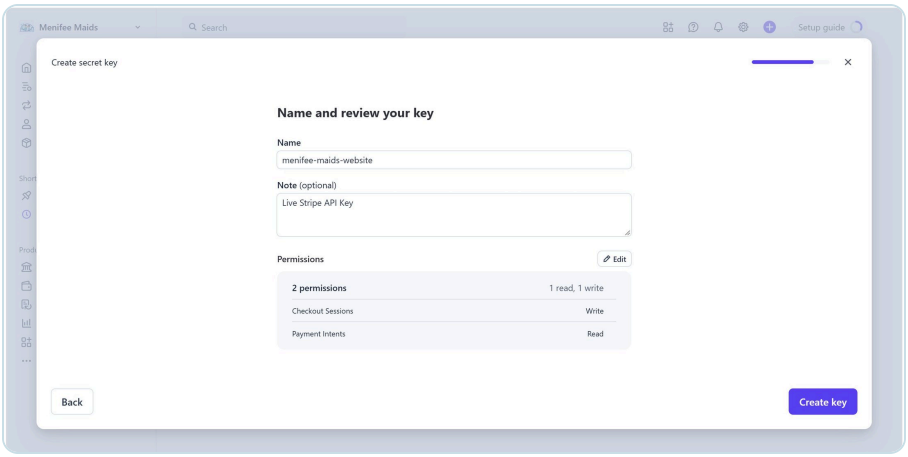
SMS opt-in recorded with a timestamp; opt-out language enforced on business texts; bilingual privacy policy; carrier registration planned before automated texting.

Cost & abuse guardrails

$5 budget alert; photo-upload and attachment limits; log ingestion caps recommended before enabling extra monitoring.

Least privilege everywhere

Each credential gets only what its job needs: the payment key creates checkouts, the Function App's identity can reach only its own data, and staff roles limit what each person can see.



Creating the live restricted key menifee-maids-website with only two permissions: Checkout Sessions (write) and Payment Intents (read).

Responsible content

Sample testimonials are flagged for replacement with genuine reviews before launch, in line with the FTC's rule on consumer reviews, and structured rating markup is deliberately left out until the reviews are real.

12 · OPERATIONS & COSTS

Operations, monitoring & costs

Background operations

JOB	SCHEDULE (PACIFIC)	PURPOSE
`sendReminders`	Daily 5:00 PM	Day-before reminder in the customer's language; marks jobs so nobody is reminded twice
`rollUpCustomers`	Nightly 2:30 AM	Builds lifetime customer history that outlives the 90-day job retention
`archiveExpiring`	1st of month 3:00 AM	JSON + CSV snapshot of expiring jobs to Blob Storage; emails the owner
`persistBooking` + poison	On queue message	Writes fallback bookings (create-only); emails full details if retries are exhausted

Monitoring: API health endpoint, Cosmos DB metrics and Data Explorer, Stripe webhook delivery logs, GitHub Actions run history and Application Insights request tracing.

Running costs

SERVICE	PLAN	COST PROFILE
Static Web Apps	Free	$0, including the custom domain, TLS and built-in auth
Cosmos DB for NoSQL	Free-tier discount, 1,000 RU/s shared	$0 within the free allowance
Azure Functions	Linux Consumption	Pay per execution, within the monthly free grant at this volume
Blob & Queue Storage	Standard	Cents per month for photos and archives
Communication Services	Email, pay as you go	A fraction of a cent per email
Stripe	Standard	Per successful card payment only, no monthly fee

Result

No fixed monthly infrastructure bill. Costs scale with real usage (bookings, emails, payments), with a budget alert as a safety net.

13 · SKILLS DEMONSTRATED

What this project demonstrates

The skills behind the result, each backed by evidence in this report.

Cloud architecture (Azure) · Serverless APIs (Azure Functions, Node.js) · NoSQL data modeling (Cosmos DB) · Offline-first sync design · Payments integration (Stripe Checkout, webhooks) · Identity & RBAC · Secrets management · CI/CD (GitHub Actions) · DNS, custom domains, SPF/DKIM · Production troubleshooting & root-cause analysis · Responsive, accessible UI · Internationalization (EN/ES) · End-to-end test automation · Technical writing

COMPETENCY	EVIDENCE IN THIS PROJECT
Cloud architecture	Designed a seven-service Azure solution around free tiers, with clear decision records and trade-offs
Full-stack development	Bilingual front end, 13-endpoint API, background functions, offline-first sync engine
Problem solving under pressure	Kept the client's business running through a platform incident by redesigning the critical path
Root-cause analysis	Isolated runtime, CI and credential faults one layer at a time; caught a silent data gap and a dead mailbox by testing assumptions
Security mindset	Least-privilege keys, managed identity, signed webhooks, HMAC-protected links
Quality engineering	130+ automated checks against real code, rerun on every release
Product thinking	Turned client feedback (control over messages, client protection, mobile use) into concrete features
Communication	Step-by-step runbooks, a professional README, a support case Microsoft could reproduce, and this report

Lessons I'm taking forward

1. **Check the support matrix first.** One unsupported runtime version stopped every background job until it was identified.
2. **Keep the critical path short.** Anything customers depend on shouldn't hinge on a background component.

1. **Verify where the data lands, not what the screen says.** The local-only dashboard looked like it worked until I checked the database.
2. **Design with the platform's limits.** Browsers can't attach files to texts, so a private link and a QR hand-off solved it properly.
3. **Document as you go.** 167 screenshots made the support case, this report and every runbook possible.

14 · WHAT'S NEXT

What's next

The platform is live and stable. These are the next improvements, ordered by business value.

1

Company texting number

One business number for every text, sent by the platform, with replies landing in an owner-only Messages inbox. Paired with hiding customer contact details from helper dashboards to protect the client base. Carrier (A2P 10DLC) registration required.

2

AI review management

After each cleaning, every customer gets a quick bilingual check-in. Azure AI Language analyzes their comments in English or Spanish, alerts the owner the moment someone is unhappy, and drafts a reply for the owner to approve. Every customer is still invited to leave a public review, so it stays honest and compliant.

3

Platform modernization

Migrate the Function App to Flex Consumption ahead of Linux Consumption's announced end of support on September 30, 2028, and move scheduled work to identity-based connections end to end.

4

Launch content & monitoring

Genuine customer reviews in place of samples, cancellation and refund terms in both languages, and alerts if a background job ever fails.

Delivery plan

Now
Next 30 days

- Real customer reviews; cancellation and refund policy
- Alerts for failed background jobs

Next
1–3 months

- Company texting number and carrier registration
- Owner-only Messages inbox for replies
- Hide customer contact details from helper dashboards

Later
3–12 months

- AI review management with owner-approved replies
- AI-drafted job summaries, approved by staff
- Flex Consumption migration, well before the 2028 deadline

Guiding principle for AI features

AI assists; the business decides. Every AI-drafted reply goes through the same human review panel used today, and the AI never decides which customers are invited to leave a review.

15 · BUILD THIS FOR YOUR BUSINESS

The same platform, built around your business

Menifee Maids is one example. The same building blocks fit any business that books appointments, sends staff into the field and gets paid per job, and every piece can be shaped to how that business actually works: cleaning, landscaping, pool care, pest control, mobile detailing, salons, tutoring, clinics and more.

1

Online booking & scheduling

Instant quotes, real availability, first and backup choices, and reminders, in one or more languages.

2

Payments & invoicing

Per-invoice card payments that reconcile themselves, with live revenue totals and exports.

3

Staff portals with roles

Owner, manager and crew views on any phone, each with exactly the access it needs.

4

Customer communication

Pre-written, reviewed messages from your own domain, with photos, links and text hand-off.

5

Data, search & reporting

A searchable record of every job and customer, with filters, exports and lifetime history.

6

Responsible AI features

Review management, drafted replies and summaries that save time while people stay in control.

Custom where it matters, scalable from day one

WHAT YOU GET	WHY IT MATTERS
Starts near $0 a month	Serverless, pay-per-use Azure services mean no servers to rent while the business is small.
Scales without a rebuild	The same architecture serves a two-person team or thousands of customers; capacity grows with usage.
Secure by default	Sign-in with roles, least-privilege keys, managed identities, and card payments that never touch the site.

WHAT YOU GET	WHY IT MATTERS
Yours to run	Automated deployments, step-by-step runbooks and plain-language documentation for the team.
Built for your customers	Your branding, your workflow and your languages, on any phone.

How an engagement works

1. **Discover.** Map how the business runs today, where time and money leak, and what success looks like.
2. **Design.** Choose the architecture and features, with costs and trade-offs explained in plain language.
3. **Build and test.** Ship in verified phases, with automated tests run against the real code.
4. **Launch.** Custom domain, email deliverability, payments live, and staff onboarded and trained.
5. **Support and grow.** Versioned releases, monitoring and a roadmap of next improvements.

Why work with me

End-to-end ownership

One accountable builder from the first conversation to production support: requirements, architecture, code, cloud, payments and launch.

Business outcomes first

Every feature traces back to a real problem: missed requests, slow payments, scattered records or a language barrier.

Production-grade from day one

Automated tests against the real code, CI/CD, role-based security and documentation the team can actually use.

Transparent costs

Architectures built on free and pay-per-use tiers, with a budget alert, so costs only grow when the business does.

Calm under pressure

When a platform component failed, I kept customers unaffected, found the root cause and worked it through with Microsoft Support.

Built for real people

Bilingual, phone-first designs that owners, staff and customers can use without training.

7	18	130+	2	$0
Azure services working together	API endpoints and background functions	automated checks on every release	languages across site, dashboards and messages	fixed monthly infrastructure cost

LET'S BUILD SOMETHING

Cloud & AI solutions that ship.

Secure, scalable Azure platforms designed around how a business actually works, and delivered end to end: discovery, architecture, build, launch and support.

FOR BUSINESS OWNERS

If your business still runs on phone calls, texts and spreadsheets, I can design and build a platform like this one around the way you work: booking, payments, staff portals, customer messaging and reporting, at a cost that grows only with your business.

FOR HIRING TEAMS

I'm pursuing cloud and AI solutions engineering roles where I can turn real customer problems into secure, scalable Azure solutions, and carry them from the first conversation through architecture, delivery and support.

Solution architecture · Azure Static Web Apps · Azure Functions · Cosmos DB · Azure AI Language · Communication Services · Stripe · GitHub Actions · Identity & RBAC · Responsible AI

Esmeralda Cabrera Ventura
Cloud & Full-Stack Developer

See it live
www.menifeemaids.com